

Computer Networks

A Practical Mental Model for Software Developers

July 2026

Contents

Part I — Foundations	2
1. What Is a Computer Network?	2
2. From Information to Signals	3
3. From Bit Streams to Frames	6
Part II — Local Networks	9
4. Ethernet and Local Delivery	9
5. Wi-Fi and the Shared Radio Medium	11
6. VLANs, Segmentation, and Switching Redundancy	14
Part III — Connecting Networks	18
7. IPv4 Addresses and Subnets	18
8. Routers, Gateways, ICMP, and Routing Tables	20
9. DHCP and Joining an IPv4 Network	24
10. IPv6 Addressing and Neighbor Discovery	27
11. Dynamic Routing and Internet Paths	31
12. NAT, Port Translation, and Firewalls	33
Part IV — End-to-End and Web Communication	37
13. Ports, Sockets, UDP, and TCP	37
14. DNS and Naming	41
15. TLS, Certificates, and Secure Connections	45
16. HTTP Requests, Responses, and Semantics	48
17. HTTP Connections and Delivery Infrastructure	52
18. From a URL to a Server Response	55
Part V — Applied Networking and Diagnosis	59
19. VPNs, Tunnels, and Private Network Overlays	59
20. Cloud, Virtual Machine, and Container Networking	62
21. Troubleshooting Networks Systematically	65
Part VI — Designing Reliable and Secure Networked Systems	69
22. Designing Reliable and Secure Networked Systems	69
23. A Durable Mental Model for Networks	72

Computer networks are one of the fundamental technologies behind modern software. Every website, mobile application, cloud service, and distributed system depends on them. Although developers interact with networks every day, many use them as a black box: requests are sent, responses arrive, and as long as everything works, there is little reason to think about what happens in between.

Understanding networking is not about memorizing protocols, standards, or packet formats. It is about developing a mental model of how computers communicate, why networks are built the way they are, and how the different technologies fit together. Once that model is established, learning new protocols and diagnosing network problems becomes significantly easier because every new concept has a place within the larger system.

This book is intended for software developers, computer science students, and junior engineers who work with networked systems but have never developed a complete mental model of how networks operate. It focuses on concepts instead of vendor-specific implementations, intuition instead of memorization, and relationships instead of isolated facts. Protocols are introduced only when they solve a problem that naturally follows from what has already been explained.

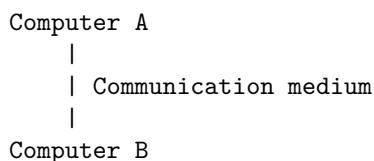
The book follows the same path that data follows. We begin with two computers connected together, gradually build a local network, connect multiple networks through routers, and eventually reach the global Internet. Along the way, we examine why each technology exists, what problem it solves, and how it cooperates with the technologies around it. By the end of the book, communication between billions of devices should feel like the result of many understandable ideas rather than a collection of unrelated mechanisms.

Part I — Foundations

1. What Is a Computer Network?

A computer network is a collection of devices capable of exchanging information. These devices may be personal computers, servers, smartphones, industrial controllers, sensors, or other systems that can send and receive data. Regardless of their purpose or performance, they all participate in the same fundamental activity: communication. Communication allows computers to use information that was created, processed, or stored elsewhere. A browser retrieves a web page from a server, a phone synchronizes photographs with cloud storage, a video call transfers audio and video in real time, and an industrial controller exchanges commands and measurements with other machines. These applications appear very different, but they depend on the same underlying ability to move information between devices.

Networks exist because computers rarely operate in isolation. Data is distributed across many systems, while users and applications expect it to remain accessible whenever it is needed. Instead of physically moving storage devices or manually copying files between machines, a network allows the information itself to travel while the participating devices remain where they are. The simplest possible network contains two computers connected through some communication medium.



Even this simple connection requires both computers to agree on several things. Information must be represented in a form that can travel through the medium, the receiver must recognize where one message begins and ends, and both sides must interpret the transmitted data in the same way. The agreed rules that make such communication possible are called protocols, but the protocols themselves are only solutions to these more fundamental problems.

As networks grow, communication becomes more complex. Exchanging data between two computers connected directly is relatively straightforward. Communication across rooms, buildings, cities, and continents introduces intermediate devices and multiple possible paths. Devices must identify destinations, determine whether they are nearby or remote, choose where to send data next, detect failures, and sometimes protect communication from observation or modification.

Modern networking solves these problems through multiple cooperating mechanisms. Local networks move data between nearby devices. Routers connect separate networks. Addressing identifies destinations independently of individual cables and switches. Transport mechanisms provide different levels of reliability, while services such as DNS allow applications to find systems by name. Security mechanisms protect communication even when the underlying network cannot be trusted.

These mechanisms are commonly described as layers, but beginning with an abstract layer model would hide the problems each layer solves. We will instead construct the network gradually. Each new concept will appear only when the system built so far can no longer solve the next communication problem. We begin with the most basic requirement: information must become something that can physically travel from one device to another.

1.1 What to Keep in Mind

A network allows independently operating devices to exchange information through agreed communication rules. Even the simplest connection requires a physical representation, message boundaries, compatible interpretation, and a way to identify where data should go. Larger networks add intermediate devices, multiple paths, reliability, naming, and security, but every later mechanism still builds on this basic exchange.

2. From Information to Signals

Computers process information as binary values. Text, images, audio, video, and application data are all represented internally as sequences of bits, where each bit has one of two possible values. The meaning of those bits depends on an agreed encoding. The same sequence may represent a number, part of a character, a color value, or a fragment of compressed audio depending on how the sender and receiver interpret it.

A network does not transmit abstract bits directly. It transmits physical changes that represent them. An electrical voltage may change along a copper conductor, pulses of light may travel through an optical fiber, or a radio transmitter may alter an electromagnetic wave. Network hardware converts bits into signals at the sender and reconstructs bits from those signals at the receiver.

Information

- > bits
- > electrical, optical, or radio signal
- > bits
- > information

This conversion requires both sides to use compatible rules. They must agree on which signal patterns represent particular values, how quickly values are transmitted, and how the receiver determines where one value ends and the next begins. These rules are part of the physical communication technology rather than decisions made separately for every application.

2.1 Transmission Media

A communication medium is the physical environment through which a signal travels. The three media encountered most often in computer networking are copper cables, optical fiber, and radio. Each transports information differently and therefore has different strengths, limitations, and typical uses. Copper cables carry changing electrical signals. They are inexpensive, easy to install, and widely used for Ethernet connections inside homes, offices, and industrial systems. Electrical signals weaken over distance and may be affected by electromagnetic interference, so cable construction, shielding, and maximum link length matter. Twisted-pair cables reduce interference by arranging conductors so that external noise affects them more evenly.

Optical fiber carries pulses of light through thin strands of glass or plastic. It supports high data rates over much longer distances than ordinary copper cabling and is resistant to electromagnetic interference. Fiber is therefore common in Internet backbones, links between buildings, data centers, and other environments where distance or capacity makes copper impractical. Its equipment and installation are generally more demanding, especially when connections must be terminated or repaired.

Radio allows communication without a physical cable between the participating devices. Wi-Fi, cellular networks, Bluetooth, satellite links, and many specialized systems all use radio waves, although they operate with different frequencies and communication rules. Radio enables mobility and convenient installation, but the medium is shared and affected by distance, obstacles, interference, and other transmitters using the same spectrum.

Copper

- > electrical signal
- > common for local wired connections

Fiber

- > optical signal
- > common for high-capacity and long-distance links

Radio

- > electromagnetic signal
- > common for wireless and mobile communication

The choice of medium affects performance and reliability, but it does not change the information being communicated. The same application data can cross several media during one journey. A laptop may send it by Wi-Fi to a home router, the router may forward it through copper Ethernet, and the Internet provider may carry it over fiber for hundreds of kilometers.

2.2 Capacity, Delay, and Actual Performance

A network link has a finite capacity, usually expressed as the number of bits it can transmit per second. This capacity is commonly called bandwidth. A link with greater bandwidth can carry more data during the same period, but bandwidth alone does not describe how quickly a particular exchange feels to the user.

Latency is the time required for data to travel from one point to another. Part of that delay comes from the physical propagation of the signal, while additional delay is introduced as devices process data and wait for busy links. A high-bandwidth connection may still have noticeable latency, and a low-latency connection may still be unable to transfer large amounts of data quickly.

Throughput is the amount of useful data that is actually transferred over time. It is usually lower than the theoretical link capacity because communication mechanisms add their own information, devices may share the medium, packets may wait in queues, and lost data may need to be transmitted again. Bandwidth is therefore the available capacity, while throughput is the result achieved under real conditions.

Jitter describes variation in delay. If a sequence of data units arrives after 20, 21, 20, and 22 milliseconds, the delay is stable. If the arrival times vary between 10 and 100 milliseconds, the average delay may still appear acceptable, but real-time audio or video can become uneven because the receiving application cannot rely on a consistent delivery pace.

Bandwidth

-> how much data the link could carry

Throughput

-> how much useful data it actually carries

Latency

-> how long delivery takes

Jitter

-> how much that delivery time varies

These properties describe different aspects of performance and should not be treated as interchangeable. Increasing bandwidth helps when large amounts of data must be transferred, but it does not automatically reduce the time required for a signal to cross a long distance. Similarly, a connection can have low latency while providing insufficient throughput for high-resolution video.

2.3 Noise, Attenuation, and Errors

Physical signals are never transmitted under perfect conditions. Electrical resistance, imperfections in optical components, obstacles, competing radio signals, and many other effects alter the signal as it travels. Attenuation is the gradual weakening of a signal over distance, while noise is unwanted energy that makes the intended signal harder to distinguish. The receiver does not need to reconstruct the original physical waveform perfectly. It only needs to decide which bit values the sender intended. Digital communication is resilient because small distortions can often be ignored as long as the received signal still falls clearly within the expected ranges. When distortion becomes too severe, however, bits may be interpreted incorrectly.

Networking technologies therefore include mechanisms for detecting damaged data. Some errors can be corrected or recovered from locally, while others cause the affected data to be discarded and transmitted again by a higher-level mechanism. We will return to error detection when we introduce Ethernet frames and to recovery when we discuss reliable transport.

2.4 Direction and Sharing

A link may support communication in one direction or both. Simplex communication carries data only one way. Half-duplex communication allows both sides to transmit, but not at the same time. Full-duplex communication allows simultaneous transmission in both directions. Modern wired Ethernet links are normally full-duplex: each connected device can send and receive independently at the same time. Wireless communication is different because nearby devices usually share the same radio channel. They must coordinate access to the medium so that their transmissions do not continuously interfere with one another. This difference will become important when we compare switched Ethernet with Wi-Fi.

A medium may also be dedicated to two devices or shared among many. A direct cable between a computer and a switch provides a separate physical link, while several wireless devices connected to one access point compete for the same radio airtime. The listed link speed therefore does not always represent capacity available exclusively to one device.

We can now transmit bits across a physical connection, but a continuous sequence of bits is not yet enough to form a practical network. The receiver must know which bits belong together, who sent them, and which nearby device should receive them. Solving those problems turns a raw link into a local network.

2.5 What to Keep in Mind

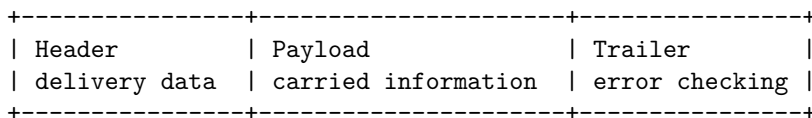
Networks carry physical signals rather than abstract bits. Copper uses electrical changes, fiber uses light, and wireless systems use radio. Bandwidth describes available capacity, throughput describes achieved transfer, latency describes delay, and jitter describes variation in that delay. Physical media introduce attenuation, interference, and errors, while duplex and sharing rules determine how devices may use a link.

3. From Bit Streams to Frames

A physical link can carry a sequence of bits, but a receiver cannot treat that sequence as one endless message. It needs to recognize which bits belong together, determine what each unit is for, and know where one transmission ends before the next begins. Networking technologies solve this by grouping data into structured units called frames.

A frame contains a limited amount of data together with information needed to deliver and validate it across one local link or network. The useful information carried for a higher layer is called the payload. Additional information placed before or after the payload allows network hardware to recognize the frame, identify its intended recipient, and detect whether it was damaged during transmission.

Frame



The exact fields depend on the networking technology, but the general structure appears repeatedly throughout networking. A header describes how a unit should be handled, the payload contains data from the layer above, and an optional trailer contains information that is easier to calculate after the payload has been processed. This pattern will later reappear in IP packets and transport-layer segments.

3.1 Recognizing Frame Boundaries

The receiver must know where a frame begins and ends. A technology may use a recognizable signal pattern, an explicit length field, reserved control values, pauses between transmissions, or a combination of these methods. The details are handled by network hardware, but the result is important: a continuous physical signal becomes a sequence of separate units that can be processed independently.

Raw transmission

...bitsbitsbitsbitsbitsbitsbitsbits...

After framing

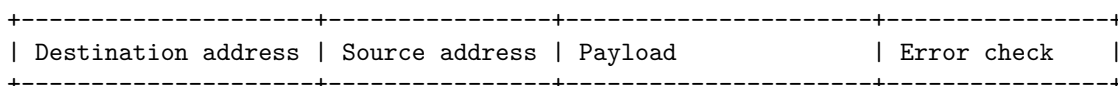
| frame 1 | frame 2 | frame 3 |

Independent frames limit the effect of errors and make a shared network manageable. If one frame is damaged, the receiver can discard that frame rather than losing synchronization with an entire stream. Devices can also alternate transmissions because each frame forms a complete local delivery attempt. Frames have a maximum practical size. Larger frames carry more useful data relative to their headers, but they occupy the link for longer and are more costly to retransmit when damaged. Smaller frames reduce that delay and recovery cost but create more overhead. Each link technology therefore defines a maximum frame size that balances efficiency with predictable operation.

The maximum amount of higher-layer data that a link can carry in one frame is commonly called its maximum transmission unit, or MTU. A typical Ethernet network uses an MTU of 1,500 bytes for the IP packet carried inside the frame. Other technologies may support different sizes. The consequences become visible when a packet must cross several links whose limits do not match, which we will examine when discussing IP.

3.2 Local Delivery Information

When more than two devices share or participate in a local network, a frame needs to indicate where it should go. The header therefore includes a destination address and usually a source address. These are link-layer addresses: they identify interfaces for delivery within the current local network rather than destinations anywhere in the world.



The destination address tells local network equipment and receiving devices which interface should accept the frame. The source address identifies the interface that transmitted it and allows replies or forwarding decisions to be made. Ethernet uses MAC addresses for this purpose, while other link technologies may use different forms of local identification. A frame may target one recipient, a selected group, or every device in the local network. These delivery forms are called unicast, multicast, and broadcast. **Unicast:** one sender to one destination; **Multicast:** one sender to an interested group; **Broadcast:** one sender to every device in the local network.

Most ordinary communication uses unicast. Broadcast is useful when the sender does not yet know the exact recipient or needs every local device to receive the message, but broadcasts consume capacity and processing time across the whole local network. Multicast provides a more selective alternative for traffic intended for a group, although applications and networks must explicitly support it.

The scope of these addresses is limited. A frame can deliver data across the current Ethernet or wireless network, but its source and destination addresses are not used to guide the data across the entire Internet. When communication crosses a router, the frame used on one link ends there, and a new frame is created for the next link.

3.3 Detecting Transmission Errors

Signals may be distorted while traveling through the medium, causing one or more received bits to differ from what was sent. Frames usually contain an error-detection value calculated from their contents. The sender calculates the value before transmission, and the receiver calculates it again from the received data. If the values do not match, the frame was altered in transit.

Ethernet uses a frame check sequence based on a cyclic redundancy check. The mathematics is not important for our mental model. What matters is that the check can detect many common forms of corruption without sending a duplicate copy of the data.

Sender

- > calculates error check
- > transmits frame and check

Receiver

- > calculates the check again
- > compares the two values
 - > equal: frame is accepted
 - > different: frame is discarded

Error detection does not necessarily provide recovery. A wired Ethernet receiver normally discards a damaged frame and leaves higher layers to recover if the missing data matters. Some link technologies, especially wireless ones, may acknowledge and retransmit frames locally because radio interference makes loss more common and local recovery can be faster than waiting for an end-to-end mechanism.

This distinction appears throughout networking. One mechanism may detect a problem, another may attempt local recovery, and a higher layer may provide complete end-to-end reliability. Repeating some responsibilities at different scopes is not always wasteful; each layer has different information and can respond to different failures.

3.4 Encapsulation

The payload of a frame is usually another structured unit rather than raw application data. When a computer sends information, each networking layer adds the information it needs around the unit produced by the layer above. This process is called encapsulation.

Application data

- > transport unit
- > IP packet
- > link-layer frame
- > physical signal

At the receiving computer, the process is reversed. The link layer validates the frame and removes its local delivery information. The network layer processes the packet inside it, the transport layer processes its own unit, and the application receives the original data. This reverse process is called decapsulation. Encapsulation allows each layer to solve a specific problem without requiring every application to understand cables, local addressing, routing, loss recovery, and signal encoding. An application provides data to the networking system, and each layer adds enough context for its part of the journey. The layers are not merely an organizational diagram; they are cooperating transformations applied to the same information.

The names used for these units describe their current role. A frame is the unit handled by a local link. A packet is the unit routed between networks. A segment commonly refers to TCP data, while a datagram is often used for UDP or for a self-contained packet more generally. In casual discussion, people often use *packet* for several of these units, but precise names help clarify which layer is making a decision. **Application:** data; **Transport:** segment or datagram; **Network:** packet; **Link:** frame. We can now turn a stream of physical

signals into separate, addressed, verifiable frames. The next step is to examine the technology that performs this local delivery in most wired networks: Ethernet.

3.5 Layer Models as Shared Vocabulary

This book has constructed the stack from concrete problems rather than beginning with a memorized model. The conventional models are still useful because documentation, tools, and infrastructure products use their names.

Book responsibility	TCP/IP model	OSI terminology
Application protocols	Application	Layers 5-7
TCP, UDP, and QUIC	Transport	Layer 4
IPv4 and IPv6	Internet	Layer 3
Ethernet and Wi-Fi	Link	Layer 2
Signals and media	Physical	Layer 1

The mapping is approximate rather than a law of nature. TLS can be described as part of the application stack or as a security layer between application protocols and transport. QUIC combines transport, security, and multiplexed streams in one protocol. Proxies terminate one application connection and create another, while tunnels add a new outer packet around an inner one.

Layer numbers are therefore shorthand for responsibility. A “Layer 2 switch” primarily forwards frames, a “Layer 3 route” uses IP prefixes, and a “Layer 4 load balancer” chooses through transport information. The useful question remains which header, state, and boundary a component is acting on.

3.6 What to Keep in Mind

Framing turns a continuous signal into independent units that can be addressed, validated, and processed. A frame contains local-delivery information, a payload, and usually an error check. Encapsulation places transport data inside an IP packet, the packet inside a link-layer frame, and the frame onto the physical medium; the receiver reverses that process. Frames belong to one local link, while packets can continue across multiple links.

Part II — Local Networks

4. Ethernet and Local Delivery

Ethernet is the dominant technology for wired local networks. It defines how devices format frames, identify local destinations, and exchange data across physical links. Modern Ethernet is used in homes, offices, industrial systems, and data centers, even though the speed, cabling, and scale of those networks may differ considerably. An Ethernet network is local in scope. Its purpose is not to choose a path across the Internet, but to move a frame between interfaces that belong to the same local network. That distinction becomes important later: Ethernet handles one local step, while IP handles the larger journey across multiple networks.

4.1 MAC Addresses

Ethernet interfaces are identified by Media Access Control addresses, usually shortened to MAC addresses. A MAC address is a fixed-length value commonly written as six hexadecimal pairs.

3C:52:82:1A:7F:09

The address identifies a network interface for Ethernet delivery. A computer with both wired Ethernet and Wi-Fi normally has a separate MAC address for each interface because each one participates independently in local communication. MAC addresses are intended to be unique, and manufacturers receive assigned address ranges so they can create addresses without coordinating every device individually. In practice, software can change or imitate a MAC address, so it should not be treated as a trustworthy identity. Its role is local forwarding, not authentication. An Ethernet frame carries both source and destination MAC addresses.

```
Computer A
MAC A
  |
  | Ethernet frame
  | source: MAC A
  | destination: MAC B
  v
```

```
Computer B
MAC B
```

The destination interface accepts frames addressed to its own MAC address, relevant multicast addresses, or the broadcast address. The Ethernet broadcast address is written as FF:FF:FF:FF:FF:FF and represents every device in the local broadcast domain.

4.2 Ethernet Switches

A switch connects multiple Ethernet links and forwards frames only where they need to go. It learns which MAC addresses are reachable through each port by observing the source address of every frame it receives.

```
      Switch
port 1 | port 2 | port 3
    |      |      |
    A      B      C
```

Suppose Computer A sends a frame through port 1. The switch reads the source MAC address and records that this address is reachable through port 1. When Computer B later sends a frame through port 2, the switch learns B's address in the same way.

```
MAC address table
MAC A -> port 1
MAC B -> port 2
MAC C -> port 3
```

When a frame arrives, the switch examines its destination MAC address. If the address is already present in the table, the frame is forwarded only through the corresponding port. Devices connected to unrelated ports do not receive it.

```
A sends to B
A
-> switch port 1
-> lookup MAC B
```

```
-> forward through port 2
-> B
```

If the destination is unknown, the switch cannot yet choose a single port. It floods the frame through every relevant port except the one on which it arrived. The destination device receives the frame, while the others discard it because the destination MAC address does not match their interfaces. The reply allows the switch to learn the previously unknown source address. Once both addresses are known, later frames can be forwarded directly. Entries eventually expire because devices may disconnect, move to another port, or be replaced.

Broadcast frames are also flooded because every device in the local broadcast domain is an intended recipient. Multicast frames may be flooded as well unless the switch has additional information about which ports contain interested receivers.

4.3 Collision Domains and Broadcast Domains

A modern switch gives each port its own collision domain. Because every connected device has a separate full-duplex link, one device can transmit without creating an electrical collision with traffic on another port. The broadcast domain is larger. A normal broadcast frame is forwarded across all switch ports that belong to the same local network. Several connected switches may therefore form one broadcast domain even though they contain many separate physical links.

```
A --- Switch 1 --- Switch 2 --- D
      |               |
      B               C
```

One broadcast domain:

A, B, C, and D receive broadcasts

This distinction explains why a switch improves local forwarding without separating the network at the IP level. It reduces unnecessary unicast traffic on individual links, but broadcasts still reach the whole local network unless an additional boundary is introduced.

4.4 What to Keep in Mind

Ethernet delivers frames inside a local network by using MAC addresses. A switch learns which source addresses are reachable through each port and forwards known unicast traffic only where required, while broadcasts and unknown destinations are flooded within the broadcast domain. Modern switched Ethernet gives each endpoint a dedicated full-duplex link, but it does not provide global routing or guaranteed delivery.

5. Wi-Fi and the Shared Radio Medium

Wi-Fi provides local network communication over radio rather than a dedicated cable. It serves the same broad purpose as Ethernet: moving frames between nearby devices and connecting them to the rest of a local network. The absence of a cable changes the physical medium, however, and that difference affects capacity, reliability, privacy, and the way devices coordinate transmission.

A typical Wi-Fi network contains an access point and one or more stations. A station is any connected wireless device, such as a laptop, phone, printer, or sensor. The access point coordinates the wireless network and normally bridges it to a wired Ethernet network.

```
Laptop
  \
Phone ---- Wi-Fi access point ---- Ethernet switch ---- Router
  /
Printer
```

The access point is not necessarily the router. Consumer devices often combine an access point, Ethernet switch, router, firewall, and other services in one enclosure, which makes the roles easy to confuse. In a larger network, access points are usually separate devices connected to switches, while routers operate at network boundaries.

5.1 Network Names and Access Points

A Wi-Fi network is advertised through a Service Set Identifier, or SSID, which is the visible network name shown by devices. The SSID helps users and software select a network, but it is not a security boundary and does not uniquely identify one physical access point. Several access points may advertise the same SSID so that one logical wireless network covers a larger area.

Each radio interface on an access point also has a more specific identifier, commonly represented by a MAC address and called a Basic Service Set Identifier, or BSSID. A device may therefore appear connected to one SSID while actually communicating through one particular access point radio.

```
SSID: Office
Access point A, BSSID A
Access point B, BSSID B
Access point C, BSSID C
```

This separation makes roaming possible. The user sees one network name, while the device moves between access points as signal conditions change. The transition is not always instantaneous because the device must decide when to move, authenticate with the new access point, and update the path through the local network. Before exchanging ordinary network traffic, a station discovers available networks, selects one, and associates with an access point. If the network is protected, authentication and encryption are established as part of joining it. Once connected, higher layers can operate much as they do over Ethernet: the device can obtain IP configuration, resolve local addresses, reach a gateway, and communicate with remote systems.

5.2 One Shared Medium

A switched Ethernet device normally has a dedicated full-duplex link to one switch port. Wi-Fi stations near the same access point instead share radio airtime. A transmission occupies the channel for every device that can hear it, even when only one destination is meant to accept the frame.

```
Switched Ethernet
A ---- dedicated link ---- Switch
B ---- dedicated link ---- Switch
Wi-Fi
A \
B >---- shared radio channel ---- Access point
C /
```

Because the medium is shared, only one nearby transmitter can use a channel successfully at a given moment. A station first listens to determine whether the channel appears busy. If the channel is available, it waits according to the Wi-Fi access rules and then transmits. If another device is already transmitting, it waits and tries later. This approach is called Carrier Sense Multiple Access with Collision Avoidance, or CSMA/CA.

Collision avoidance differs from the collision detection used by older shared Ethernet. A radio cannot reliably listen for another transmission while sending its own much stronger signal, so Wi-Fi tries to reduce the probabil-

ity of collisions before they happen. Collisions can still occur when two devices make similar decisions or cannot hear one another. After receiving a normal unicast frame, the destination sends a short acknowledgment. If the sender does not receive that acknowledgment, it assumes the frame may have been lost and retries. These local acknowledgments and retransmissions improve reliability over an unpredictable radio medium, but they consume additional airtime. **Station:** transmit frame; **Access point:** acknowledge; **No acknowledgment:** wait and retransmit.

A frame that requires several attempts occupies the channel several times. This is one reason a weak or noisy device can affect more than its own connection: its slower transmissions and retries consume airtime that other devices could have used.

5.3 Airtime, Data Rate, and Throughput

A Wi-Fi connection may advertise a high physical data rate, but that number is not equivalent to application throughput. Management traffic, acknowledgments, encryption, contention, waiting, and retransmissions all use part of the channel. Several devices also share the available airtime rather than each receiving the full listed capacity. A device close to the access point can usually use a faster and more efficient radio encoding. As distance, obstacles, or interference make the signal harder to distinguish, the devices select a more robust but slower rate. This rate adaptation keeps the connection usable at the cost of capacity.

Strong, clear signal

- > faster radio rate
- > less airtime per frame

Weak or noisy signal

- > slower radio rate
- > more airtime per frame
- > greater chance of retries

The slowest device does not simply set one fixed speed for the entire network, but slow transmissions occupy the shared channel for longer. A distant client sending the same amount of data may therefore consume much more airtime than a nearby client. This is also why adding access points can improve capacity even when coverage already appears adequate. If clients are distributed among access points using suitable channels, each group competes for a smaller share of airtime. Adding access points without planning channels or transmit power can instead increase interference and make the network worse.

5.4 Channels, Frequency Bands, and Interference

Wi-Fi operates in defined radio frequency bands divided into channels. Nearby networks using the same or overlapping channels must share airtime or interfere with one another. A device may show a strong signal from its access point while still achieving poor throughput because many other transmitters occupy the channel. Lower frequency bands generally travel farther and penetrate obstacles more effectively, while higher bands provide more spectrum and can support greater capacity over shorter distances. The exact channel widths and capabilities vary across Wi-Fi generations, but the practical trade-off remains stable: **Lower frequency:** generally longer reach, fewer available channels, and often more congestion; **Higher frequency:** generally shorter reach, more available spectrum, and potentially greater capacity.

Wider channels can carry more data under good conditions, but they also occupy more spectrum and leave fewer independent channels for neighboring access points. In a crowded office or apartment building, several narrower well-planned channels may provide better total performance than one very wide channel surrounded by interference. Interference is not limited to other Wi-Fi networks. Bluetooth devices, some wireless peripherals, microwave ovens, and other radio systems may operate in overlapping spectrum. Walls, floors, metal structures, people, and reflections also change how signals travel. Wireless performance therefore depends on the environment in ways that wired Ethernet usually does not.

5.5 Wireless Security

Radio transmissions can be received by devices that are not physically connected to the network. Wi-Fi therefore needs encryption and authentication at the local link rather than relying on the difficulty of reaching a cable. Modern protected networks commonly use WPA2 or WPA3. In a personal network, devices often authenticate using one shared passphrase. The passphrase helps establish encryption keys, but the keys used for actual traffic are derived rather than transmitted directly as the password. WPA3 improves several weaknesses of older password-based methods, particularly resistance to offline guessing attacks.

Enterprise Wi-Fi commonly authenticates individual users or devices through an external identity service rather than one shared password. This allows credentials to be revoked separately, access to be audited, and different policies to be applied without changing one secret on every device. An open network provides no link-layer encryption between the station and access point. HTTPS and other end-to-end encryption can still protect application traffic, but unencrypted or poorly protected protocols remain exposed. Even on a protected Wi-Fi network, link encryption covers only the wireless hop. Traffic is decrypted by the access point before continuing through the wired network.

Laptop

-> encrypted Wi-Fi link

Access point

-> ordinary local network forwarding

Router

-> Internet

Server

Wi-Fi security should therefore be understood as local-link protection, not as a replacement for TLS, VPNs, application authentication, or authorization.

5.6 Wi-Fi and Ethernet Together

Wi-Fi and Ethernet normally form one cooperating local network. A wireless frame received by an access point can be bridged into an Ethernet frame and forwarded by switches. The IP packet carried inside remains the same even though the local frame format changes.

Laptop

-> Wi-Fi frame

Access point

-> Ethernet frame

Switch

-> Ethernet frame

Router

The access point therefore behaves partly like a bridge between two link technologies. It learns or tracks which wireless clients are associated with it, receives frames over radio, and forwards their payloads into the wired network. Replies follow the reverse path. From the application's perspective, the transport may be invisible. The same connection can operate over wired Ethernet or Wi-Fi because IP and the higher layers sit above both. Performance and reliability still reveal the difference: Wi-Fi has shared airtime, variable radio rates, interference, roaming, and local retransmissions, while switched Ethernet normally provides a dedicated and predictable link.

Our local network can now include wired and wireless devices, but all of them still belong to one broad broadcast domain. As the network grows, unrestricted broadcasts, security requirements, and organizational boundaries make one flat local network increasingly undesirable. The next chapter divides a physical switching infrastructure into separate logical networks using VLANs.

5.7 What to Keep in Mind

Wi-Fi provides local frame delivery over shared radio airtime. Stations associate with an access point, contend for the channel, acknowledge unicast frames, and adapt their radio rate as conditions change. Signal strength alone does not determine performance; interference, channel use, retries, client distance, and access-point placement all matter. Wi-Fi encryption protects the wireless hop, while end-to-end security remains the responsibility of higher layers.

6. VLANs, Segmentation, and Switching Redundancy

A switched network can connect many devices efficiently, but by default those devices still belong to one broad local network. Broadcast frames reach every connected segment, devices can communicate directly at the link layer, and one failure or configuration mistake can affect a large area. As networks grow, it becomes useful to divide one physical switching infrastructure into several smaller logical networks.

A Virtual Local Area Network, or VLAN, creates such a division. Devices in different VLANs can use the same physical switches and cables while remaining in separate broadcast domains. From the perspective of Ethernet forwarding, each VLAN behaves like an independent local network.

One physical switch

```
+-----+
| VLAN 10: Engineering |
|   Laptop A          |
|   Workstation B      |
|                       |
| VLAN 20: Administration |
|   Laptop C           |
|   Printer D          |
+-----+
```

A frame in VLAN 10 is not forwarded into VLAN 20 simply because both devices are attached to the same switch. Communication between the VLANs requires a router or another device capable of network-layer forwarding. VLANs therefore separate local Ethernet communication while routers connect the resulting networks intentionally.

6.1 Why Segment a Local Network?

A small home network can often operate as one broadcast domain without difficulty. An office, factory, campus, or data center may contain hundreds or thousands of devices with different responsibilities and security requirements. Keeping every device in one local network creates several problems. Broadcast and unknown-destination traffic reaches a larger number of devices. This traffic is not necessarily dominant, but its scope grows with the network. More importantly, every device shares one local trust boundary. A compromised workstation may directly reach printers, cameras, controllers, administrative systems, and servers unless higher-level controls prevent it. Segmentation creates explicit boundaries. **Users:** VLAN 10; **Servers:** VLAN 20; **Guest Wi-Fi:** VLAN 30; **Industrial devices:** VLAN 40.

The division may follow organizational roles, security requirements, traffic patterns, physical locations, or operational responsibilities. A guest wireless network should not normally share unrestricted local access with employee computers. Industrial controllers may need tightly limited communication with selected management systems. Servers may accept traffic only through firewalls or load balancers rather than directly from every workstation.

VLANs do not provide complete security by themselves. They create separation at the link layer, but traffic can still be routed between them according to the configured rules. The value lies in making that communication explicit and controllable rather than allowing every device to reach every other device locally.

6.2 Access Ports

A switch port connected to an ordinary device is commonly configured as an access port. The port belongs to one VLAN, and frames sent or received by the attached device are treated as part of that VLAN.

```
Laptop A
  |
  | access port, VLAN 10
  |
Switch
```

The laptop normally does not need to know that a VLAN exists. It sends and receives ordinary Ethernet frames. The switch associates those frames with VLAN 10 because of the port configuration.

```
Switch port 1 -> VLAN 10
Switch port 2 -> VLAN 10
```

```
Switch port 3 -> VLAN 20
Switch port 4 -> VLAN 30
```

Devices connected to ports 1 and 2 can exchange frames directly because they belong to the same VLAN. Devices on ports 3 and 4 belong to different broadcast domains even though they are attached to the same physical switch. Broadcasts are confined accordingly. **Broadcast from port 1:** forwarded to VLAN 10 ports, reaches port 2, and does not reach VLAN 20 or VLAN 30. The switch maintains forwarding information separately for each VLAN. The same MAC address should not ordinarily appear in several places, but the forwarding decision is still made within the context of a particular VLAN.

6.3 Trunk Links

A connection between switches often needs to carry traffic for several VLANs. Using one physical cable for each VLAN would be wasteful and difficult to scale. A trunk link carries frames from multiple VLANs over one physical connection.

```
Switch A
  |
  | trunk: VLANs 10, 20, 30
  |
Switch B
```

The receiving switch must know which VLAN each frame belongs to. Ethernet provides this information through an additional VLAN tag inserted into the frame. The commonly used tagging method is defined by IEEE 802.1Q.

```
Ordinary Ethernet frame
+-----+-----+-----+
| Destination | Source       | Payload |
+-----+-----+-----+
Tagged frame on a trunk
+-----+-----+-----+-----+
| Destination | Source       | VLAN tag | Payload |
+-----+-----+-----+-----+
```

The tag contains a VLAN identifier and a small amount of control information. A switch receiving the tagged frame uses the identifier to process it within the correct logical network. When the frame leaves through an ordinary access port, the tag is normally removed before delivery to the endpoint.

```
Laptop A
-> untagged frame
Switch A
-> associates frame with VLAN 10
-> adds VLAN 10 tag on trunk
Switch B
-> reads VLAN 10 tag
-> removes tag on access port
Laptop B
```

A trunk is usually configured to carry only the VLANs that are needed across that link. Allowing every VLAN everywhere increases the effect of mistakes and makes the network harder to understand. Some endpoints can process tagged frames directly. Servers, hypervisors, firewalls, and specialized appliances may connect through trunk ports when one physical interface must participate in several VLANs. Ordinary user devices are more commonly attached through access ports.

6.4 VLANs and Subnets

A VLAN defines a link-layer broadcast domain. An IP subnet defines a group of IP addresses that can communicate directly without passing through a router. In most practical designs, one VLAN corresponds to one IP subnet.

```
VLAN 10
-> subnet 192.168.10.0/24
VLAN 20
-> subnet 192.168.20.0/24
```

VLAN 30

```
-> subnet 192.168.30.0/24
```

These are different concepts even when they align. The VLAN determines which Ethernet frames can be exchanged locally. The subnet determines which IP destinations a host considers directly reachable. Keeping one subnet per VLAN makes the boundaries consistent. Devices in VLAN 10 treat addresses in the VLAN 10 subnet as local and send other destinations to a router. Devices in VLAN 20 do the same for their own subnet.

Placing one IP subnet across several disconnected VLANs creates a mismatch: hosts believe destinations are local, but Ethernet broadcasts and address resolution cannot reach them. Placing several unrelated subnets in one VLAN is technically possible but often makes the network harder to reason about. The usual alignment is therefore intentional rather than accidental. We will examine subnet calculation and routing later. For now, the useful relationship is: **VLAN**: local Ethernet boundary; **Subnet**: local IP boundary; **Router**: communication between subnets.

6.5 Inter-VLAN Routing

Devices in different VLANs cannot exchange ordinary Ethernet frames directly. To communicate, they send IP packets to a router that has access to both VLANs.

VLAN 10

```
-> router
-> VLAN 20
```

The router receives a frame from VLAN 10, removes the local Ethernet header, examines the destination IP address, and decides that the packet belongs in VLAN 20. It then creates a new Ethernet frame suitable for VLAN 20.

Computer A in VLAN 10

```
-> frame addressed to router
```

Router

```
-> removes VLAN 10 frame
-> routes IP packet
-> creates VLAN 20 frame
```

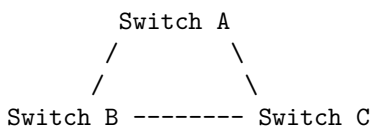
Computer B in VLAN 20

This is a complete local example of encapsulation across multiple links. The IP packet remains logically the same while each local network uses its own frame. Inter-VLAN routing can be performed by a dedicated router, a firewall, or a multilayer switch. A multilayer switch combines high-speed Ethernet switching with IP routing. The device name matters less than the responsibility: link-layer forwarding stays within a VLAN, while network-layer forwarding connects VLANs.

Routing also provides a place to apply policy. A router or firewall can permit users to reach a web server while preventing them from connecting directly to a database. Guest devices may be allowed to reach the Internet but denied access to internal VLANs. **Guest VLAN**: Internet: allowed, Employee VLAN: denied, and Server VLAN: denied. The VLAN creates the boundary; the routing or firewall policy decides what may cross it.

6.6 Switching Loops

Redundant links improve availability because traffic can continue if one cable or switch fails. In a plain Ethernet network, however, connecting switches through multiple active paths can create a loop.



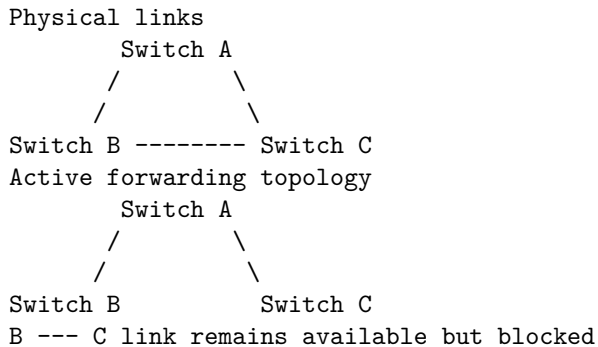
Ethernet frames do not contain a general hop limit that forces them to expire after crossing several switches. A broadcast or unknown-destination frame can therefore circulate repeatedly through the loop. Suppose Switch A floods a broadcast toward both B and C. B forwards it toward C, while C forwards another copy toward B. The switches receive repeated copies and continue flooding them. Each copy may also be learned as if the source MAC address moved between ports. **Broadcast frame**: copied around loop, copies multiply, links become saturated, and MAC address table becomes unstable.

The resulting broadcast storm can consume most available capacity and make the local network unusable. This can happen quickly because switches forward frames at hardware speed. A switching loop is different from having

several legitimate routed paths across the Internet. Routers process packets using network-layer information and reduce a hop-limit value. Ethernet switching within one broadcast domain lacks that same protection.

6.7 Spanning Tree

The Spanning Tree Protocol prevents switching loops by placing selected redundant links into a non-forwarding state. The switches exchange control messages, choose a common logical root, and calculate one active loop-free path through the network.



The physical redundancy remains present, but not every path forwards ordinary frames at the same time. If an active link fails, the switches can recalculate the topology and activate a previously blocked path. **Normal operation:** redundant path blocked; **Link failure:** topology recalculated and alternate path begins forwarding. Modern networks commonly use faster variants such as Rapid Spanning Tree Protocol, but the underlying purpose remains the same: preserve redundancy without allowing loops.

Spanning Tree is often invisible until something fails or is misconfigured. A blocked port may look wasteful because the cable exists but carries no ordinary traffic. Its value is that the network can activate it when the active path disappears.

6.8 What to Keep in Mind

A VLAN divides one switching infrastructure into separate link-layer broadcast domains. Access ports place ordinary devices into one VLAN, trunks carry several VLANs with tags, and routing is required when traffic crosses from one VLAN and subnet to another. Redundant Ethernet paths must be controlled because frames have no general hop limit; Spanning Tree blocks loops while preserving a path that can be activated after failure.

Part III — Connecting Networks

7. IPv4 Addresses and Subnets

Ethernet and Wi-Fi identify interfaces for delivery inside one local network, but MAC addresses do not describe where a device belongs in a larger topology. Internet Protocol adds logical addresses that can be grouped into hierarchical prefixes and carried across many different links. IPv4 uses 32-bit addresses, normally written as four decimal octets such as 192.168.10.42.

An address is assigned to an interface, not to a computer as one indivisible object. A laptop can have one address on Wi-Fi and another on Ethernet, while a router necessarily has addresses on several interfaces because it participates in several networks. Addresses can also change as devices move. They describe network locations more than permanent identity.

7.1 Prefixes and CIDR Notation

An address becomes useful for routing when it is paired with a prefix length:

192.168.10.42/24

The /24 says that the first 24 bits identify the network and the remaining 8 bits identify an address inside that network. All addresses whose first 24 bits match belong to 192.168.10.0/24. This is Classless Inter-Domain Routing, or CIDR, notation. A shorter prefix fixes fewer bits and therefore describes a larger block. A longer prefix describes a smaller block.

10.0.0.0/8

-> large block

192.168.10.0/24

-> smaller block

192.168.10.0/28

-> smaller again

IPv4 prefixes can also be written as subnet masks. /24 corresponds to 255.255.255.0, but CIDR notation communicates the boundary more directly and is preferred in modern documentation. Prefixes are not merely a way to count devices. They let routers describe groups of destinations with one route. A route for 192.168.10.0/24 represents the complete block instead of listing every address separately.

7.2 Local and Remote Destinations

Before sending a packet, a host compares the destination with its configured prefixes. A host using 192.168.10.42/24 treats 192.168.10.80 as local because the first 24 bits match. It can deliver the packet directly over the current Ethernet or Wi-Fi network after resolving the destination's local link address. A destination such as 203.0.113.25 does not match the local prefix. The host sends that packet to a router, normally through its default route. **Destination matches local prefix:** deliver on the local link; **Destination does not match:** send to a router.

This decision does not require the host to know the complete path. It only decides whether the next receiver is the final local destination or a gateway that can forward the packet farther.

7.3 Network, Broadcast, and Host Addresses

In an ordinary IPv4 subnet, the address with every host bit set to zero represents the network. In 192.168.10.0/24, that value is 192.168.10.0. The address with every host bit set to one is the directed broadcast address, 192.168.10.255. Ordinary host addresses lie between them.

Network: 192.168.10.0

Hosts: 192.168.10.1 through 192.168.10.254

Broadcast: 192.168.10.255

Specialized point-to-point designs can use small blocks differently, but the normal multi-host model is sufficient for understanding local networks and address planning.

7.4 Public, Private, and Special Addresses

Public IPv4 addresses must be globally unique and routable through the public Internet. Private ranges are intended for reuse inside separate networks:

10.0.0.0/8
172.16.0.0/12
192.168.0.0/16

A home and an office can both use 192.168.1.10 without conflict as long as their networks remain separate. Problems appear when two overlapping private networks must be connected through a VPN, cloud peering, or a company merger. Routers then cannot distinguish which copy of the prefix is intended. Private does not mean secure, and public does not mean openly accessible. These words describe routing scope. A public service may be protected by a firewall, while a private subnet may contain untrusted devices.

The loopback range keeps traffic inside the current host. 127.0.0.1 is commonly used for services that should be reachable only locally. Binding a server to loopback is therefore different from binding it to an external interface or to all interfaces. IPv4 link-local addresses come from 169.254.0.0/16. A device may assign itself one when it has joined the physical network but cannot obtain normal configuration through DHCP. Seeing such an address often means that Ethernet or Wi-Fi works while IP configuration does not.

7.5 Subnetting and Address Planning

A larger block can be divided by extending its prefix. A /24 can, for example, become four /26 networks:

192.168.10.0/26
192.168.10.64/26
192.168.10.128/26
192.168.10.192/26

Longer prefixes create more networks with fewer addresses in each. Tools calculate boundaries reliably, so ordinary developers do not need to memorize subnetting tables. The important skill is reading the prefix and understanding the resulting scope. Address plans should leave room for growth and avoid needless overlap. Hierarchical allocation also enables route aggregation. An organization can advertise one broad prefix for a region while using smaller subnets internally.

7.6 Interface Configuration and Duplicate Addresses

A usable IPv4 configuration normally contains an address and prefix, a default gateway, and DNS resolver addresses. Static configuration is useful for routers, infrastructure, and selected servers, but it creates manual work and duplicate-address risk. DHCP supplies the same values dynamically and is therefore preferred for most clients.

Two active interfaces using the same address in one local network create an ambiguous network-layer destination even though their MAC addresses remain different.

```
Computer A
-> 192.168.10.42
Computer B
-> 192.168.10.42
Result
-> ambiguous local delivery
```

Neighbor mappings may alternate between the devices, causing intermittent or misdirected traffic. Operating systems often probe for duplicates when assigning an address, and DHCP servers track their leases, but manual addresses must still be coordinated with the dynamic pool. On Linux, `ip -brief address` confirms interface state and assigned prefixes; Chapter 21 combines that evidence with routes, neighbors, DNS, sockets, and packet capture.

7.7 What to Keep in Mind

An IPv4 address belongs to an interface and is interpreted together with a prefix. The prefix defines which destinations are local and which require a router. Public addresses are globally routed, private ranges can be reused in isolated networks, loopback remains inside the host, and link-local addressing often indicates failed automatic configuration. Subnetting extends prefixes to create smaller networks, while good address planning avoids overlap and supports route aggregation.

8. Routers, Gateways, ICMP, and Routing Tables

A local link can deliver frames only within its own Ethernet or Wi-Fi network. Communication between IP networks requires a router: a device or software function with interfaces in more than one network that can receive a packet on one side and forward it on another. **Laptop network:** router and server network. A gateway is a broader term for a system through which traffic leaves one context. In ordinary host configuration, the default gateway is the router used when no more specific route exists. A home gateway often combines routing, switching, Wi-Fi, firewalling, NAT, DHCP, and DNS forwarding, but these remain separate responsibilities even when one enclosure performs all of them.

8.1 Routing Tables and Next Hops

Every IP-capable host maintains a routing table. Each entry associates a destination prefix with an outgoing interface, a next-hop router, or both.

Destination	Next step
192.168.10.0/24	directly through wlan0
127.0.0.0/8	loopback
0.0.0.0/0	gateway 192.168.10.1

A connected route appears when an interface is configured with an address and prefix. If a router has 192.168.10.1/24 on one interface and 10.20.0.1/24 on another, it knows both networks are directly attached and can route between them. A static route is entered deliberately:

```
172.16.0.0/16
-> next hop 10.20.0.2
```

Static routes are predictable and useful in small or stable networks. They become difficult to maintain when many routers, paths, and failures are involved, which is why large networks use dynamic routing protocols. The default route is written as 0.0.0.0/0. Because it fixes no bits, every IPv4 destination matches it, but any more specific route wins. A workstation commonly points its default route at the local router, and that router may point its own default route toward an Internet provider.

A route can identify an outgoing interface, a next-hop router, or both. **Directly connected route:** outgoing interface; **Remote route:** next-hop router and outgoing interface used to reach it. Suppose a router has this entry:

```
172.16.0.0/16
-> next hop 10.20.0.2
```

The router must also know how to reach 10.20.0.2. Usually that address belongs to a connected network.

```
10.20.0.0/24
-> directly connected through interface B
```

The two entries cooperate.

```
Destination packet:
  172.16.5.20
```

```
Route:
  send to next hop 10.20.0.2
```

```
Connected route:
  reach 10.20.0.2 through interface B
```

This recursive reasoning ends at a directly connected link, where the router can create a frame addressed to the next hop's local link address. An invalid configuration may point to a next hop that itself has no route. In that case the remote route exists on paper but cannot be used to transmit a frame. Routing tables must eventually resolve to an operational outgoing interface.

8.2 Longest-Prefix Matching and Route Preference

Several routes can match one destination. The router selects the route with the longest matching prefix—the most specific available information.

```
10.0.0.0/8      -> path A
10.20.0.0/16    -> path B
```

```
10.20.30.0/24    -> path C
0.0.0.0/0        -> default path
```

Destination 10.20.30.42 matches all four entries, but /24 wins. Destination 10.20.80.5 uses /16, while an unrelated public address uses the default route. This allows broad summaries and narrow exceptions to coexist. Longest-prefix matching is performed before comparing metrics between routes for the same prefix. A low metric cannot make a /8 override an available /24 match. A route must eventually resolve to a usable outgoing interface. If it points to next hop 10.20.0.2, the system also needs a connected or otherwise resolvable route to that next-hop address.

Several routes may exist for the same prefix. The system then needs a way to prefer one.

```
10.20.0.0/16
-> router A, metric 100
10.20.0.0/16
-> router B, metric 200
```

A lower metric often represents the preferred route, although the precise comparison depends on the operating system and routing source. Metrics can reflect administrative preference, path cost, interface speed, routing-protocol calculations, or manually chosen priorities. They are not a universal physical measurement. A metric of 100 in one routing system is not directly comparable to 100 in another. Some routers install several equal-cost routes and distribute traffic among them.

```
10.20.0.0/16
-> path A, equal cost
-> path B, equal cost
```

Distribution is often based on a hash of packet or flow attributes so that one conversation remains on a stable path. Sending successive packets of one stream through paths with very different delays can cause reordering. Redundant routes improve availability only if failures are detected and the preferred path is updated. A route can remain configured even when the remote destination beyond its next hop is unavailable. Dynamic protocols and active monitoring provide better failure awareness than a simple static entry.

8.3 From an IP Next Hop to a Local Frame

Routing chooses an IP next hop, but Ethernet and Wi-Fi need a local link address. IPv4 uses the Address Resolution Protocol, or ARP, to map a local IP address to a MAC address. For a local destination, the host broadcasts a question such as:

```
Who has 192.168.10.80?
Tell 192.168.10.42.
```

The owner replies with its MAC address. For a remote destination, the host does not resolve the Internet server's MAC address. It resolves the default gateway instead.

```
IP packet destination:
  203.0.113.25
Ethernet frame destination:
  local router MAC
```

Hosts cache recently learned mappings in a neighbor table. Entries age because devices move, interfaces change, and addresses can be reassigned. ARP provides no authentication, so local networks should not treat an address-to-MAC claim as trusted identity. IPv6 uses Neighbor Discovery rather than ARP, but the responsibility is similar: turn a selected on-link next hop into information suitable for local delivery.

Broadcasting an ARP request before every frame would be inefficient. Hosts and routers therefore keep recently learned mappings in an ARP cache, also called a neighbor table.

```
192.168.10.1  -> 3C:52:82:10:00:01
192.168.10.80 -> 3C:52:82:10:00:50
```

Entries expire or change state because devices may disconnect, replace an interface, or move an address. The operating system can refresh a mapping when it is used again. A stale entry can temporarily send frames to the wrong MAC address. Operating systems include mechanisms to verify or relearn neighbors, but the exact timing differs. Clearing the neighbor cache can sometimes resolve a local address change during troubleshooting, although doing so without understanding the cause may only hide a recurring problem.

ARP has no built-in authentication. A device can claim that another IP address belongs to its own MAC address. This enables ARP spoofing attacks in which traffic is redirected through an attacker or disrupted. Switch features, network segmentation, endpoint protections, and encrypted higher-layer protocols reduce the risk, but ARP itself assumes a reasonably trusted local network. This is another reason a VLAN should not automatically be treated as a safe environment merely because it is internal.

8.4 Forwarding One Packet

Assume a laptop at 192.168.10.42/24 sends a packet to server 10.20.0.25/24 through a router whose interfaces are 192.168.10.1 and 10.20.0.1. The laptop sees that the destination is remote, selects its default route, and resolves the gateway's MAC address. It sends:

```
Frame 1
  source MAC: laptop
  destination MAC: router interface A
Packet
  source IP: 192.168.10.42
  destination IP: 10.20.0.25
```

The router removes Frame 1, performs a route lookup, and finds that 10.20.0.0/24 is directly connected. It resolves the server's MAC address and creates Frame 2.

```
Frame 2
  source MAC: router interface B
  destination MAC: server
Packet
  source IP: 192.168.10.42
  destination IP: 10.20.0.25
```

The frame addresses change because each frame belongs to one local link. The packet addresses normally remain the same across ordinary routing. NAT can intentionally rewrite them later.

8.5 ICMP, Hop Limits, and Path MTU

Internet Control Message Protocol accompanies IP with information about delivery problems and network conditions. **ping** uses ICMP echo messages, but diagnostics are only one part of its role. Routers and hosts can report that a destination is unreachable, that a packet's lifetime expired, or that a packet is too large for the next link.

IPv4 packets contain a Time to Live value that routers use as a hop limit. Every router reduces it by one, and a router discards the packet and commonly returns an ICMP error when it reaches zero. This prevents a routing loop from circulating one packet forever. **traceroute** uses the same behavior deliberately by sending probes with gradually increasing hop limits and observing which router reports each expiration. The hop limit bounds a packet's lifetime; it does not prevent a loop from causing loss or congestion before the packet expires.

Links can have different MTUs. A packet that fits one link may be too large for another. IPv4 can fragment packets in some circumstances, but modern systems prefer to discover a usable path size and avoid fragmentation. IPv6 routers never fragment packets; they return an ICMPv6 Packet Too Big message and the sender reduces its size.

```
Sender
-> packet too large for later link
Router
-> ICMP Packet Too Big
Sender
-> reduces packet size
```

This is Path MTU Discovery. TCP Maximum Segment Size limits TCP payload near connection establishment, but MSS is not the same as link MTU because network and transport headers also consume space. Blocking every ICMP message can therefore break valid traffic. A characteristic symptom is that small requests succeed while larger transfers stall.

8.6 Forwarding and Routing

Forwarding is the per-packet data-plane operation: **Receive packet:** inspect destination, select route, and transmit through next interface. Routing is the control-plane process that creates and maintains the informa-

tion forwarding uses. Connected interfaces, static configuration, and dynamic routing protocols all contribute candidate routes. Several equal or alternative routes can exist for the same prefix. Metrics and protocol preference determine which are installed, and equal-cost routes may distribute different flows across several paths. Keeping one flow on a stable path usually reduces packet reordering. Redundancy helps only when the network detects failures and updates forwarding. A static route can remain present even when the destination beyond its next hop is unreachable.

8.7 Reading the Decision on Linux

The most useful Linux command is often not the complete table but a direct question:

```
ip route get 203.0.113.25
```

A result can show the chosen gateway, interface, and source address. `ip neighbor` shows the local mappings used to construct frames. These tools expose two separate decisions: where the packet should go and which local interface should receive the next frame.

8.8 What to Keep in Mind

A routing table maps destination prefixes to connected interfaces or next-hop routers. Hosts and routers use longest-prefix matching, with a default route only when nothing more specific exists. ARP resolves the selected local IPv4 next hop to a MAC address. Each router removes one local frame, forwards the packet, and creates a new frame for the next link. ICMP reports delivery conditions, hop limits prevent indefinite loops, and Path MTU Discovery prevents packet-size mismatches from becoming silent failures.

9. DHCP and Joining an IPv4 Network

An IPv4 client normally needs an address, prefix, default gateway, and DNS resolver addresses before applications can communicate. Entering those values manually on every laptop, phone, virtual machine, and temporary device would be slow and error-prone. The Dynamic Host Configuration Protocol, or DHCP, supplies them automatically for a limited period called a lease. A DHCP server manages one or more address pools. It may run on a home router, a centralized company server, or a cloud networking service. DHCP does not replace IP addressing; it automates the configuration IP requires.

9.1 Discover, Offer, Request, Acknowledge

The initial exchange is commonly summarized as DORA:

Client	DHCP server
----- Discover ----->	
<----- Offer -----	
----- Request ----->	
<----- Acknowledge -----	

The client initially has no usable address and may not know the server's address, so the Discover is sent as a local broadcast. A server responds with an offer containing a proposed address and options such as the prefix, gateway, DNS resolvers, and lease duration. The client requests one offer, and the selected server acknowledges the lease.

Several servers may respond. Coordinated redundancy can be intentional, while an unauthorized server can supply a false gateway or DNS resolver. DHCP does not inherently know which server is legitimate from an organizational perspective, so enterprise switches may restrict where server replies are accepted.

9.2 Leases, Renewal, and Reservations

A lease allows the client to use an address for a defined period. The client normally attempts renewal before expiration, first with the original server and later more broadly if necessary. If the lease expires without renewal, the client must stop using the address because the server may assign it elsewhere.

Dynamic assignment does not imply frequent change. A returning client can request its previous address, and the server grants it when the address remains appropriate and available.

```
Previous address:
    192.168.10.42
Client reconnects:
    -> requests 192.168.10.42
Server:
    -> grants it if still valid
    -> offers another address otherwise
```

A reservation associates a known client with a preferred address while retaining centralized management. Servers may recognize clients through a client identifier or the MAC address visible on the local network, depending on the implementation. A changed interface, randomized MAC address, or cloned virtual machine can therefore appear as a new client. Remembering an old address does not grant permission to keep using it; the current DHCP exchange confirms that the address has not been assigned elsewhere.

9.3 Pools, Conflicts, and Exhaustion

A pool contains the addresses available for ordinary leases. It can exclude ranges reserved for routers, servers, or fixed infrastructure.

```
Subnet:
    192.168.10.0/24
Dynamic pool:
    192.168.10.100 through 192.168.10.220
```

When the pool is exhausted, existing clients may continue using current leases while new clients fail to obtain normal configuration. They may show a 169.254.x.x link-local address or report that Wi-Fi is connected without Internet access. The link can therefore work while DHCP fails one stage later. Exhaustion can result from a pool that is too small, leases that are unnecessarily long, stale reservations, unexpected growth, or unauthorized devices. Very short leases improve reuse but increase renewal traffic.

A server tracks which addresses it believes are available, but its knowledge may be incomplete. An administrator may have configured a static address without excluding it from the DHCP pool, or a device may be using an address incorrectly. Before fully accepting a lease, some clients test whether the address already appears in use. They may send an ARP probe or perform another duplicate-address check.

Offered address:

192.168.10.42

Client checks:

Is another device already using it?

If a conflict is detected, the client can send a DHCP Decline message. The server marks the address as problematic and avoids offering it again until the condition is investigated or a timeout expires. **Client:** DHCP Decline: address appears occupied; **Server:** remove address from available pool. This mechanism reduces the effect of inconsistent configuration, but it is not a substitute for proper address planning. False or malicious conflict reports can also waste pool capacity.

9.4 DHCP Relay

Initial DHCP broadcasts do not cross routers, but organizations often centralize the service. A relay agent on the client subnet receives the broadcast and forwards it through ordinary routed communication. **Client VLAN:** local DHCP broadcast; **Router or relay:** unicast to central server; **DHCP server:** selects pool for the originating subnet. The relay identifies the client network so the server can offer an address from the correct pool. This serves many VLANs without extending one broadcast domain across the organization.

A missing or incorrect relay configuration can make DHCP fail in one VLAN while it works everywhere else. The physical link, switching, and general routing can all be healthy even though this required service path is absent.

9.5 Joining Home and Enterprise Networks

A laptop joining protected home Wi-Fi first discovers the SSID, authenticates, and associates with an access point. At that point it can exchange local frames but still lacks normal IPv4 configuration. DHCP then supplies, for example:

Address: 192.168.1.42/24

Gateway: 192.168.1.1

DNS: 192.168.1.1

The operating system installs a connected route for 192.168.1.0/24 and a default route through 192.168.1.1. Before sending remote traffic, it resolves the gateway's MAC address through ARP. When an application uses a hostname, the configured resolver supplies the destination IP address. **Link becomes available:** DHCP configuration, routes installed, gateway neighbor resolved, DNS resolves application name, and first application packet leaves. An enterprise Ethernet client follows the same network-layer sequence, although switch authentication, VLAN assignment, and DHCP relay may occur before the lease is issued.

A wired office device follows a similar process with different link setup.

Laptop

-> Ethernet link becomes active

Switch

-> places access port in VLAN 10

Laptop

-> sends DHCP broadcast

Relay

-> forwards request to central server

DHCP server

-> assigns VLAN 10 configuration

9.6 Common Failure Patterns

No address is assigned when the server is unavailable, the pool is exhausted, the relay is missing, the client belongs to the wrong VLAN, or local broadcasts cannot reach the service. An assigned address with no remote access usually points instead to an incorrect prefix, missing gateway, routing failure, or firewall rule. Numeric destinations working while names fail suggests that address assignment and routing succeeded but DNS configuration did not. Only new clients failing strongly suggests pool or lease-management problems. One client

receiving inconsistent configuration can indicate a changing client identifier, movement between VLANs, or several DHCP servers.

The stable diagnostic questions are simple: which address and prefix were assigned, which default route was installed, which DNS resolvers were supplied, and when does the lease expire? Chapter 21 applies these questions with system tools and logs.

9.7 What to Keep in Mind

DHCP supplies the address, prefix, gateway, DNS resolvers, and lease information required for ordinary IPv4 use. The client discovers servers through local broadcast, chooses an offer, and receives an acknowledged lease. Renewal allows addresses to remain stable while still returning unused capacity to the pool. Reservations provide predictable addresses under central control, relays serve remote subnets, and pool exhaustion or VLAN mismatches can leave the link connected while higher-layer communication fails.

10. IPv6 Addressing and Neighbor Discovery

IPv6 solves the same network-layer problem as IPv4: it gives interfaces logical addresses and lets routers forward packets between networks. Its most visible change is a much larger 128-bit address space, but it also redesigns local discovery and automatic configuration. IPv6 does not replace Ethernet, Wi-Fi, transport protocols, DNS, TLS, or firewalls; it changes how the network layer identifies destinations and finds local neighbors.

10.1 Address Notation and Prefixes

IPv6 addresses are written as eight groups of four hexadecimal digits:

```
2001:0db8:0042:0000:0000:0000:0000:0025
```

Leading zeroes inside a group may be removed, and one continuous sequence of zero groups may be replaced by `::`:

```
2001:db8:42::25
```

The compression marker can appear only once because otherwise the number of omitted groups would be ambiguous. IPv6 uses CIDR prefix notation just as IPv4 does. A typical LAN uses `/64`:

```
2001:db8:42:10::25/64
```

The first 64 bits identify the subnet and the remaining 64 bits identify an interface within it. IPv6 has enough address space that ordinary subnets do not need to conserve addresses through tiny prefixes. `/64` is also expected by several automatic configuration mechanisms.

10.2 Address Scopes

A global unicast address is intended for normal routed communication and is conceptually similar to a public IPv4 address. Documentation examples commonly use `2001:db8::/32`, which is reserved for that purpose. Every IPv6 interface also creates a link-local address from `fe80::/10`. Link-local addresses are valid only on the current link and are essential for router discovery and neighbor communication. Because the same value can exist on several interfaces, commands sometimes include an interface scope such as `fe80::1%wlan0`.

The loopback address is `::1`, while `::` is the unspecified address used before a sender has selected a usable address. Unique local addresses from `fc00::/7` provide private organizational addressing, commonly from the `fd00::/8` portion, but they are not a direct replacement for good global IPv6 planning. IPv6 does not use broadcast. Multicast addresses beginning with `ff` deliver packets to defined groups such as all nodes or all routers on a link. Avoiding broadcast reduces the need for every host to inspect traffic that is irrelevant to it.

10.3 Neighbor Discovery and Local Delivery

IPv6 replaces ARP with the Neighbor Discovery Protocol, which uses ICMPv6. Neighbor Solicitation and Neighbor Advertisement messages discover the link-layer address associated with an on-link IPv6 address. **Neighbor Solicitation:** Who owns this IPv6 address?; **Neighbor Advertisement:** This is my link-layer address. The solicitation is sent to a solicited-node multicast group derived from the target address rather than to every device on the link. The sender stores the result in its neighbor cache and verifies reachability over time. Neighbor Discovery also supports router discovery, duplicate-address detection, and redirects. ICMPv6 is therefore not merely optional diagnostic traffic; filtering it indiscriminately can break fundamental IPv6 operation.

IPv6 still runs over Ethernet, Wi-Fi, and other link technologies. The encapsulation model does not change.

Application data: TCP or UDP, IPv6 packet, Ethernet or Wi-Fi frame, and physical signal. For a local destination, NDP resolves the destination's link-layer address. For a remote destination, the host sends the frame to the router's MAC address while the IPv6 packet retains the final destination.

Frame

Destination MAC:

 local router

IPv6 packet inside

Destination IPv6:

 remote server

At each routed hop, the link-layer frame changes while the IPv6 source and destination normally remain the same. **Local frame addresses:** change at every hop; **IPv6 addresses:** remain end to end. Ordinary IPv6 routing therefore follows the same layered logic established for IPv4.

10.4 Router Advertisements, SLAAC, and Address Privacy

IPv6 routers periodically send Router Advertisements and answer Router Solicitations from new hosts. An advertisement can identify the default router, announce on-link prefixes, provide lifetimes, and indicate which configuration method clients should use. With Stateless Address Autoconfiguration, or SLAAC, a host combines an advertised /64 prefix with an interface identifier to form an address.

Advertised prefix:

2001:db8:42:10::/64

Generated address:

2001:db8:42:10:7c1a:8f2d:91b4:3301

Early systems sometimes derived the interface identifier from the MAC address. Modern operating systems usually create opaque identifiers and may keep both a stable address for one network and temporary privacy addresses for outbound connections.

2001:db8:42:10:8f2a:10c5:91d4:2b70

-> stable address

2001:db8:42:10:5c8e:3f1a:7b20:aa14

-> temporary address

Temporary addressing makes passive correlation harder but does not provide anonymity; DNS, accounts, cookies, TLS identifiers, and traffic patterns still reveal continuity. Servers normally need stable addressing or stable DNS registration.

Before using any newly created or assigned address, the host performs Duplicate Address Detection through Neighbor Discovery. It sends a solicitation for the tentative address and makes the address usable only when no conflict appears. DAD also applies to manually configured and DHCPv6-provided addresses. It detects accidental reuse but is not strong authentication, because a hostile local device can falsely claim an address.

A client can therefore join an IPv6 network without a stateful address server:

Link becomes available

-> link-local address is created

-> router and prefix are discovered

-> global address is generated

-> duplicate check succeeds

-> default route is installed

10.5 DHCPv6 and DNS Configuration

DHCPv6 can provide addresses, DNS resolvers, and other options. Networks may use it statefully for address assignment, statelessly only for additional configuration, or not at all when Router Advertisements and another DNS mechanism are sufficient. IPv6 deliberately separates router discovery from DHCPv6. A client normally learns the default router through Router Advertisements rather than through a DHCPv6 gateway option. This differs from IPv4 DHCP, where one response often supplies address, prefix, gateway, and DNS together. The precise combination depends on network policy: **SLAAC**: address and default route; **DHCPv6 or Router Advertisement options**: DNS and additional configuration.

10.6 Dual Stack and IPv4 Interoperation

Many systems run IPv4 and IPv6 simultaneously. DNS can return both **A** and **AAAA** records, and the client chooses a reachable address according to platform policy and observed connection progress.

api.example.com

-> AAAA 2001:db8:100::25

-> A 203.0.113.25

A modern client may begin with IPv6 and start IPv4 shortly afterward if the first path does not progress. Dual stack therefore means two independent routes, listener configurations, firewall policies, and failure modes. A service can work through IPv4 while failing through IPv6 because the server is not listening on ::, an IPv6 route is missing, or only the IPv4 firewall path was configured.

IPv6-only access networks can reach IPv4-only services through DNS64 and NAT64. DNS64 synthesizes an IPv6 destination from an IPv4 record, and a NAT64 gateway translates the resulting connection.

```
IPv6 client
-> synthesized IPv6 destination
-> NAT64 gateway
-> IPv4 server
```

The server sees an IPv4 connection from the translation system rather than a native IPv6 connection from the client. Applications that embed literal IPv4 addresses or assume IPv4-specific behavior can still fail. Translation preserves compatibility during migration; it is not a substitute for native IPv6 support.

10.7 Packet Headers and Path MTU

An IPv6 packet has a fixed 40-byte base header. Several fields present in the IPv4 header were removed, renamed, or moved into extension headers. Important fields include:

```
Version
Traffic Class
Flow Label
Payload Length
Next Header
Hop Limit
Source Address
Destination Address
```

The Hop Limit serves the same practical purpose as IPv4 TTL. Every forwarding router reduces it by one, and the packet is discarded when it reaches zero.

```
IPv4:
    Time to Live
IPv6:
    Hop Limit
```

The Next Header field identifies the transport protocol or an IPv6 extension header. Optional features are arranged as a chain rather than making the base header variable in the same way as IPv4 options. **IPv6 base header:** extension header, extension header, TCP or UDP, and application data. Routers generally process the base header efficiently and examine only the extension information relevant to forwarding or policy. IPv6 routers do not fragment packets in transit. If a packet is too large for the path, the router returns an ICMPv6 Packet Too Big message. The sender adjusts its packet size. **Sender:** sends packet; **Router:** path cannot carry it and returns Packet Too Big; **Sender:** reduces packet size.

This makes Path MTU Discovery especially important and is another reason ICMPv6 must not be blocked indiscriminately.

10.8 Security and Practical Failure Patterns

IPv6 generally removes the need for address-sharing NAT, but it does not remove the need for firewalls. A globally unique address can be routed directly while stateful policy still rejects unwanted inbound traffic. Dual stack means two real routes, listener configurations, firewall policies, and monitoring paths. Enabling IPv6 without equivalent controls can create an unintended path around an IPv4-only policy.

Failure evidence should be interpreted by stage. An interface with only a link-local address may not have received a usable Router Advertisement. A global address without a default route permits local but not remote communication. Numeric IPv6 destinations working while names fail suggests DNS configuration, while an AAAA record reaching a service that is not listening on IPv6 causes failure only for clients that select that address. Small exchanges working while larger transfers stall points toward blocked ICMPv6 Packet Too Big messages and failed Path MTU Discovery.

Useful Linux commands include `ip -6 address`, `ip -6 route`, `ip -6 neighbor`, and `ip -6 route get <destination>`. They establish local configuration and route selection; Chapter 21 combines them with DNS, transport, TLS, and application evidence.

10.9 What to Keep in Mind

IPv6 uses 128-bit addresses and commonly assigns /64 subnets. Interfaces create link-local addresses, routers advertise prefixes and default-route information, SLAAC generates addresses, and DHCPv6 can provide stateful or supplemental configuration. Neighbor Discovery replaces ARP and also supports router discovery and

duplicate detection. IPv6 removes broadcast and usually removes address-sharing NAT, but routing, DNS, transport, and firewall responsibilities remain. Dual-stack systems must operate and secure both protocol families independently.

11. Dynamic Routing and Internet Paths

Static routes work well when topology is small and stable. As networks grow, every path change would otherwise require manual updates on many routers. Dynamic routing protocols let routers exchange reachability information, select usable paths, and adapt when links or neighboring routers fail. The protocol builds the control-plane view. Forwarding remains the data-plane operation that applies the resulting route to each packet.

11.1 Convergence and Route Selection

A network converges when participating routers have processed a topology change and reached a stable view of the available paths. During convergence, some packets may follow old paths, loop, or be dropped. **Link fails:** routers detect change, advertisements propagate, routes are recalculated, and forwarding stabilizes. Fast convergence improves availability, but overly sensitive failure detection can cause route flapping when an unstable link repeatedly appears and disappears. Timers, thresholds, and hold-down behavior balance quick recovery against stability.

A router may learn the same prefix from connected interfaces, static configuration, and several protocols. It first chooses among candidate routes using administrative preference and protocol rules, then applies longest-prefix matching when forwarding packets. Metrics are meaningful only inside the routing method that defines them.

Not every router needs detailed knowledge of every remote prefix. A branch office with one connection to headquarters can use a default route toward that connection. **Branch subnets:** branch router and default route to headquarters. Headquarters advertises or statically configures routes back to the branch prefixes. The branch is a **stub network** because traffic does not normally pass through it to reach other networks. A dual-connected branch may learn two defaults and prefer one while retaining another for failure. **Primary default:** provider A; **Backup default:** provider B.

The design must still ensure that return traffic can reach the branch through the active path. An outbound route alone does not create bidirectional reachability. Internet providers and large transit networks cannot rely on one simple default in the same way. They need detailed external routes so they can choose among many peers, customers, and upstream networks. The appropriate amount of routing information depends on the role of the device.

11.2 Interior Routing and OSPF

Inside one organization, routers commonly use an interior routing protocol to exchange reachability and adapt to topology changes. In a link-state protocol, routers advertise their links and neighbors, build a shared topology view, and independently calculate preferred paths. Open Shortest Path First, or OSPF, is a widely used example. Its path costs commonly reflect interface bandwidth or deliberate administrative choices.

Large OSPF deployments can divide the routing domain into areas so every router does not need every internal detail. Area 0 connects the other areas, and boundaries can summarize routes. The practical lesson for a developer is not how to design those areas, but why an internal route can change after a link or router fails while the destination address remains the same. OSPF areas control routing scale; they are not security zones and do not replace VLANs or firewalls.

Distance-vector protocols provide a useful contrast. Instead of building a complete topology map, a router learns which destinations its neighbors claim to reach and at what cost.

Link-state

-> routers calculate from a topology view

Distance-vector

-> routers compare reachability reported by neighbors

Older protocols such as RIP illustrate the model through hop counts and slower convergence. They require protections against accepting a neighbor's outdated route as an independent path. The implementation details vary, but the shared problem is distributed agreement after change.

11.3 Autonomous Systems, Peering, and BGP

The public Internet is divided into autonomous systems, each operated under one routing policy and identified by an autonomous-system number. Border Gateway Protocol, or BGP, exchanges reachable IP prefixes between them. BGP is a path-vector and policy protocol rather than a pure shortest-path algorithm. An advertisement can include the sequence of autonomous systems through which the route passed, helping prevent loops and supporting policy decisions.

Prefix:
203.0.113.0/24
AS path:
64520 64510

Networks exchange traffic through transit and peering relationships. A transit provider offers reachability to the wider Internet, while peers primarily exchange traffic for their own customers. Internet exchange points provide shared infrastructure where many networks can establish those relationships, but each participant still chooses which routes to advertise and accept.

An organization may prefer a customer route over a peer route, a peer route over paid transit, or one provider over another for resilience. The selected path can therefore contain more router or autonomous-system hops while still matching business policy. A content provider may also announce more-specific routes near users while retaining a broader aggregate elsewhere. The path observed by one client is consequently the result of several independent policies rather than one globally calculated shortest route.

11.4 Aggregation, Redundancy, and Asymmetry

Route aggregation represents several specific networks with one broader prefix.

```
203.0.112.0/24
203.0.113.0/24
-> aggregate 203.0.112.0/23
```

Aggregation reduces table size and hides internal detail. More-specific routes can still override the aggregate when necessary. Routers may install several equal-cost paths and distribute flows across them. Redundancy improves capacity and failure tolerance, but the forward and return paths do not need to be identical. BGP decisions are directional, and each network chooses routes independently. Asymmetric routing is normal until a stateful firewall, NAT device, or other intermediary requires both directions to pass through the same state. Then a path that is valid according to routing alone can still fail operationally.

11.5 Routing Failures and Application Effects

A missing route produces immediate inability to forward. A black hole exists when a route is present but traffic is discarded later. A routing loop circulates packets until their hop limit expires, while route flapping repeatedly changes paths and can cause intermittent loss. `traceroute` or `tracert` can reveal where hop-limit responses appear, but a silent router may still forward ordinary traffic, and equal-cost routing can expose different intermediate hops between tests.

Applications do not interact with OSPF or BGP directly. They open sockets toward destination addresses and experience only the resulting behavior: a brief failure during convergence, greater latency on a backup path, different forward and return paths, or reachability that differs between regions or providers. Routing redundancy also does not make an established connection immortal. TCP may recover when endpoint addresses remain valid, while a stateful firewall, NAT mapping, VPN tunnel, or mobile path change can still break the connection.

Applications should therefore use finite timeouts, propagate cancellation, reconnect long-lived sessions, retry only operations that are safe to repeat, and expose separate timing for DNS, connection establishment, TLS, and server processing. Those mechanisms respond to uncertainty; they do not implement routing inside the application.

11.6 What to Keep in Mind

Dynamic routing lets routers learn and update reachability instead of relying only on manual entries. Link-state protocols such as OSPF calculate internal paths from a shared topology view, while BGP exchanges prefixes between autonomous systems according to policy. Convergence takes time, aggregation reduces routing detail, equal-cost paths distribute flows, and forward and return paths may differ. Forwarding uses the selected table one packet at a time; applications experience only the resulting delay, loss, or path change.

12. NAT, Port Translation, and Firewalls

Private IPv4 addresses can be reused inside separate networks but are not routed across the public Internet. Network Address Translation, or NAT, rewrites packet addresses as traffic crosses a boundary. Home and business networks commonly combine NAT with stateful firewalling, but translation and security are different responsibilities.

12.1 Source NAT, Port Translation, and Address Mappings

Assume an internal laptop uses 192.168.1.42 and the router has public address 198.51.100.20. When the laptop opens a connection to 203.0.113.25:443, the router rewrites the source:

```
Before translation
192.168.1.42:51500
-> 203.0.113.25:443
After translation
198.51.100.20:62001
-> 203.0.113.25:443
```

The router records a mapping so replies addressed to 198.51.100.20:62001 can be restored to 192.168.1.42:51500. Using transport ports lets many internal connections share one public address. This is commonly called Port Address Translation. The mapping key includes protocol and endpoint information, so the same numeric port can be used independently by TCP and UDP or by unrelated external destinations. NAT modifies packets and therefore must update affected checksums. It also creates state with finite lifetime. A router cannot translate an arbitrary inbound reply unless it can associate that traffic with an existing mapping or an explicit rule.

Port translation is not the only form of NAT. A static one-to-one mapping can associate one public address with one internal address.

```
198.51.100.21
<-> 192.168.1.50
```

This preserves a stable public identity for the internal system but consumes one public address. A dynamic address pool can temporarily assign one of several public addresses to internal clients.

```
Private clients
-> public pool
    198.51.100.20
    198.51.100.21
    198.51.100.22
```

Unlike port-overloaded NAT, a pool may assign a whole public address to one active mapping. The exact behavior depends on the configured translation type. Cloud platforms and enterprise firewalls often expose these ideas through terms such as public IP association, egress NAT gateway, inbound NAT rule, or floating IP. The implementation may be distributed across virtual routers rather than one physical box, but the packet transformation is conceptually similar.

12.2 Inbound Connections, Hairpin NAT, and Carrier-Grade NAT

An external client cannot normally initiate a connection to an internal private host because no translation state exists. Port forwarding creates a destination-NAT rule:

```
198.51.100.20:443
-> 192.168.1.50:443
```

The public address and port identify the edge mapping; the router rewrites the destination and forwards the packet inward. Replies are translated back so the client continues to see the public endpoint. Successful publication requires every layer to agree: **Public route:** NAT rule, internal route, firewall permission, listening process, and valid return path. Forwarding to the wrong internal address, binding the server only to loopback, or blocking the port on the host firewall can all produce the same user-visible failure. Carrier-Grade NAT adds another translation layer inside an Internet provider. Customers share public IPv4 addresses, so ordinary inbound port forwarding may be impossible unless the provider offers a public address or a separate publication mechanism.

An internal client may try to reach an internal server through the network's public address.

Laptop:
192.168.1.42
Public service address:
198.51.100.20
Internal server:
192.168.1.50

Without special handling, the router may not apply its external destination-translation rule to traffic arriving from the inside. Hairpin NAT, also called NAT loopback or reflection, allows the packet to leave the internal client, match the public mapping, and return to another internal host.

192.168.1.42
-> 198.51.100.20:443
-> NAT reflection
-> 192.168.1.50:8443

The NAT device may also translate the internal client's source so the server's reply returns through the same translator. Otherwise, the server might reply directly to the laptop, creating an asymmetric path that does not match the state expected by the NAT device or client. An alternative is split DNS. Internal clients resolve the service name directly to the private address, while external clients receive the public address.

Inside network:
app.example.com -> 192.168.1.50
Outside network:
app.example.com -> 198.51.100.20

Both designs can work. Hairpin NAT keeps one public answer but depends on reflection support, while split DNS avoids the translation for internal traffic but requires consistent name management.

Internet providers may place many customers behind another layer of translation, called carrier-grade NAT or CGNAT.

Laptop
192.168.1.42
-> home router
100.64.12.8
-> provider NAT
198.51.100.20
-> Internet

The range 100.64.0.0/10 is reserved for shared address space used in provider networks. The home router's external address may come from this range rather than being a public Internet address. The packet can therefore be translated twice.

192.168.1.42:51500
-> 100.64.12.8:62001
-> 198.51.100.20:31002

CGNAT conserves public IPv4 addresses but complicates inbound hosting, peer-to-peer connectivity, abuse tracing, and geolocation. A customer cannot create a normal port-forwarding rule on the provider's translator. Several customers may appear to an Internet service as the same public address. Logs that record only the address may therefore be insufficient to identify one connection; the public source port and exact time can also be necessary. A user can suspect CGNAT when the router's reported external address differs from the address observed by an Internet service or when the router receives an address from private or shared space.

12.3 NAT Traversal and Application Behavior

NAT is easiest for protocols whose endpoint information remains in IP and transport headers. A protocol that embeds a private address or port inside its application payload can advertise an endpoint that a remote peer cannot reach.

Packet header:
translated public endpoint
Application payload:
private endpoint 192.168.1.42

Real-time and peer-to-peer applications therefore use traversal mechanisms. A client can contact an external discovery service to learn the public address and port assigned by its NAT. Two peers then attempt coordinated outbound traffic so both translators create compatible mappings, a broad technique known as hole punching. NAT behavior differs between devices, so direct communication is not guaranteed. When it fails, the application relays traffic through a publicly reachable server. Protocols in the ICE, STUN, and TURN family support this process for voice, video, games, and similar systems.

Peer A

- > discovery and connection attempt
- > direct path when mappings permit
- > relay when they do not

Peer B

Translation state also has a lifetime. A quiet TCP or UDP flow may appear open to the application after a NAT or firewall has forgotten its mapping. Periodic protocol traffic, application heartbeats, or transport keepalives can preserve state, but intervals should reflect failure-detection needs, intermediary timeouts, bandwidth, and battery cost. UDP mappings commonly expire more quickly because the network device cannot observe a connection lifecycle.

The durable lesson is that private addressing changes connection establishment and introduces state outside both applications. A route can exist while the required translation mapping does not, and an apparently established application session can outlive the network state that once carried it.

12.4 NAT Is Not a Firewall

NAT often blocks unsolicited inbound traffic as a side effect because no mapping exists, but it does not define a complete security policy. It does not authenticate users, authorize application operations, inspect business meaning, or guarantee that translated traffic is safe. IPv6 demonstrates the distinction clearly. Hosts can have globally routable addresses without address-sharing NAT, while a stateful firewall still permits established replies and rejects unwanted inbound connections. **NAT:** rewrites addressing; **Firewall:** decides whether traffic is allowed. The two are commonly implemented together but should be designed and diagnosed separately.

12.5 Stateless and Stateful Firewalls

A stateless firewall evaluates each packet independently using values such as source and destination address, protocol, port, and interface. A stateful firewall also tracks flows so that return traffic for an allowed connection can be recognized. **Outbound TCP connection allowed:** connection state created; **Inbound reply:** matches established state and allowed. UDP has no handshake, so stateful devices approximate a flow through endpoint information and timers. ICMP errors can be associated with the traffic that caused them and permitted as related communication.

Rules are normally evaluated in a defined order, followed by a default policy. A default-deny design explicitly permits required communication and rejects everything else. Rule direction matters: input affects traffic delivered to the firewall host, output affects traffic created by it, and forwarding affects traffic crossing through it.

12.6 Host, Network, and Zone Policy

A perimeter or cloud firewall protects a network boundary. A host firewall protects one machine even when surrounding network controls are absent or misconfigured. Using both provides separate enforcement points. Policy should describe the required path narrowly: **Internet:** TCP 443 on public proxy: allowed; **Proxy:** application port: allowed; **User subnet:** database port: denied. An address being private, internal, or behind NAT is not sufficient authorization. Compromised internal systems still need to be contained through segmentation and application-level identity.

Firewalls often group interfaces or networks into zones with different trust assumptions. **Internet:** untrusted zone; **User VLAN:** internal client zone; **Server VLAN:** protected service zone; **Management VLAN:** administrative zone. Policy then describes allowed communication between zones. **Users:** public web servers: allowed and databases: denied; **Administrators:** management interfaces: allowed; **Internet:** published HTTPS service: allowed and internal clients: denied.

Direction matters. Allowing users to initiate HTTPS connections toward the Internet does not necessarily allow Internet hosts to initiate connections toward users. Stateful filtering permits the replies to user-initiated flows while denying unrelated inbound attempts. A firewall can operate between VLANs, at the Internet edge, on an

individual host, in a cloud virtual network, or inside an application platform. Several layers may apply policy to the same connection.

12.7 Failure Patterns and Inspection

Outbound connections failing can indicate a missing route, exhausted NAT state, blocked egress rule, or unreachable destination. Inbound publication failing can indicate the public route, destination translation, internal route, host firewall, listener, or return path. Intermittent failures under load may point to connection-tracking or port exhaustion rather than bandwidth. On Linux, `ss` shows active and listening sockets, `nft list ruleset` shows nftables policy, and `conntrack -L` can display tracked flows when the tool is installed. Packet captures on both sides of a translator reveal whether addresses and ports changed as expected.

12.8 What to Keep in Mind

NAT rewrites network and transport endpoints as packets cross a boundary. Port translation lets many private IPv4 clients share one public address, while destination translation and port forwarding publish selected internal services. Translation relies on state that can expire or become exhausted. Firewalls are separate policy mechanisms: stateless rules evaluate packets, while stateful rules recognize established and related traffic. Public reachability requires routing, translation, firewall permission, a listening process, and a valid return path.

Part IV — End-to-End and Web Communication

13. Ports, Sockets, UDP, and TCP

IP delivers packets to an interface, but a host may run many networked processes at the same time. The transport layer identifies the intended application endpoint and defines how data should be exchanged. The two most common Internet transports are UDP, which sends independent datagrams, and TCP, which presents a reliable ordered byte stream.

13.1 Ports, Binding, and Sockets

A port is a 16-bit number associated with a transport protocol. A server commonly listens on a predictable destination port such as TCP 443 for HTTPS, while a client uses a temporary ephemeral source port selected by the operating system. TCP port 443 and UDP port 443 are independent endpoints.

```
Client:
    192.168.1.42:51500
Server:
    203.0.113.25:443
Protocol:
    TCP
```

The destination IP address gets a packet to the host. The transport protocol and destination port let the operating system select the receiving socket. A socket is the operating-system object through which an application uses that endpoint. A TCP server binds a listening socket and accepts connections; every accepted connection receives its own socket while the listener remains available for more clients. A flow is commonly distinguished by source address, source port, destination address, destination port, and protocol. NAT devices, firewalls, and load balancers track similar tuples.

Binding determines which local addresses are eligible. 127.0.0.1:8080 accepts only IPv4 loopback traffic, a specific interface address accepts traffic addressed to that interface, and 0.0.0.0:8080 normally accepts traffic for every suitable local IPv4 address. IPv6 uses corresponding forms such as [::1]:8080 and [::]:8080, although whether an IPv6 wildcard also accepts IPv4-mapped connections depends on platform and socket configuration.

```
Loopback binding
    -> same host only
Specific-address binding
    -> one local address
Wildcard binding
    -> all suitable local addresses
```

Binding is only one requirement for reachability. A host or network firewall can still block the port, routing can fail, and NAT may provide no public mapping. A service bound only to loopback can therefore work locally while appearing unavailable everywhere else.

The client's ephemeral port distinguishes its return traffic from other local connections. After TCP closes, the same endpoint combination may remain temporarily unavailable while old packets expire. Heavy clients, proxies, and NAT gateways can exhaust available ports or connection state when they create new connections faster than those resources become reusable. Connection pooling and multiplexed protocols reduce that pressure.

13.2 UDP Datagrams and Use Cases

UDP sends self-contained datagrams. The transport header adds source and destination ports, a length, and a checksum, but it does not establish a connection or guarantee delivery. **Application message:** one UDP datagram and one IP packet or fragments. UDP does not guarantee that a datagram arrives, arrives once, or arrives in order. It also does not automatically retransmit, regulate sender speed, or adapt to congestion. Those properties must be unnecessary or provided by the application protocol.

This makes UDP suitable for small request-response exchanges, real-time media, service discovery, and transports such as QUIC that implement their own reliability above UDP. DNS commonly uses UDP for ordinary queries because a lost query can simply be repeated. Voice and video may prefer timely delivery over waiting for a late packet to be retransmitted. A UDP server binds to a local port and receives datagrams from many remote endpoints through one socket. Applications may still maintain logical peer state, but that state belongs to the application rather than to a UDP connection handshake.

Datagram size matters. A large UDP message can require IP fragmentation, and losing one fragment prevents reconstruction of the complete datagram. Practical protocols therefore keep datagrams within a safe path size or perform their own segmentation.

UDP is suitable when one or more of these properties matter:

- Low setup overhead
- Independent messages
- Application-controlled timing
- Tolerance for occasional loss
- Multicast or broadcast
- Custom reliability

Examples include:

- DNS queries
- Voice and video media
- Online games
- Telemetry
- Time synchronization
- Service discovery
- QUIC transport

Real-time media may prefer a recent audio packet over a retransmitted old one. A game may send a newer position update that makes an older lost update irrelevant. DNS can retry a query when no response arrives. UDP does not make an application automatically fast. The application must still avoid congestion, limit message sizes, handle loss, and protect itself against spoofing or amplification abuse.

13.3 TCP Connections and the Three-Way Handshake

TCP begins with a handshake that establishes state and initial sequence numbers at both endpoints.

Client		Server
----- SYN	----->	
<----- SYN, ACK	-----	
----- ACK	----->	

The handshake confirms that packets can travel in both directions and that each endpoint is prepared to maintain the connection. A firewall can allow the SYN, a server can reject it with RST when no listener exists, or silence can cause retransmission and eventual timeout. After establishment, TCP presents an ordered byte stream. It does not preserve application message boundaries.

Application writes:

- 100 bytes
- 200 bytes

Receiver may read:

- 50 bytes
- 250 bytes

Applications therefore need their own framing, such as a length field, delimiter, fixed format, or higher-level protocol like HTTP. One write is not guaranteed to equal one read or one network packet.

13.4 Sequence Numbers, acknowledgments, and Retransmission

TCP numbers bytes so the receiver can place data in order and detect gaps. acknowledgments report the next byte expected, and selective acknowledgment can identify non-contiguous ranges already received.

Sender transmits bytes 1000-1499

Receiver acknowledges 1500

If acknowledgment progress does not occur within the calculated retransmission time, the sender sends the missing data again. Duplicate packets are recognized through sequence numbers and not delivered twice to the application. TCP's reliability is end to end between transport stacks. A successful TCP acknowledgment proves that bytes reached the remote TCP implementation, not that the remote application processed them or committed a database transaction.

Ordered delivery also creates head-of-line blocking. If bytes in the middle of the stream are missing, later bytes wait even if their packets arrived. This is required for TCP's stream abstraction. QUIC avoids this blocking between independent streams, although ordering still applies within each stream.

13.5 Flow Control, Congestion Control, and Path Capacity

Flow control protects the receiver. The receiver advertises how much more data it can buffer, and the sender limits unacknowledged bytes accordingly. **Receiver window shrinks:** sender slows; **Receiver window reaches zero:** sender waits for space. Congestion control protects the network. TCP infers available path capacity from acknowledgments, delay, and loss, then adjusts how much data it sends before waiting. A new connection normally begins cautiously and increases its sending rate as successful delivery provides evidence of capacity.

The amount of data required to fill a path is related to bandwidth multiplied by round-trip time. High-capacity long-distance links need many bytes in flight to achieve full throughput. Small receive windows, loss, or application-level limits can prevent that even when raw bandwidth is high. Flow control, congestion control, and application backpressure solve different problems: the receiver's buffer, the network path, and the application's processing capacity.

A sender cannot fill a fast long-distance path with only a small amount of data in flight. The required amount is related to the bandwidth-delay product.

Path capacity:
 100 megabits per second
Round-trip time:
 100 milliseconds
Data required in flight:
 about 10 megabits

If the sender waits for acknowledgment after only a tiny window, the link sits idle for much of each round trip. TCP window scaling allows modern connections to advertise windows larger than the original small field could represent. This matters for high-capacity, high-latency paths such as intercontinental transfers. Latency and bandwidth therefore interact. **High bandwidth + low latency:** moderate data in flight can fill path; **High bandwidth + high latency:** much larger window needed. Application design also matters. A protocol that performs many sequential request-response turns can remain slow on a high-bandwidth path because every step waits for another round trip.

13.6 Closing Connections and Detecting Failure

TCP closes each direction independently. One endpoint sends FIN when it has no more bytes to transmit, while the other can acknowledge it, finish sending remaining data, and close its own direction later. Application APIs may expose this as a half-close. A reset, or RST, terminates the connection immediately and can occur when no process listens on the destination port, an application aborts the socket, or a device rejects unexpected state.

The endpoint that actively closes can enter `TIME_WAIT`. This prevents delayed segments from an old connection from being confused with a later connection using the same endpoints and allows the final acknowledgment to be retransmitted. Many `TIME_WAIT` sockets can therefore represent normal short-lived connection churn rather than a fault, although connection reuse reduces their number.

An established connection can remain silent even after the peer or path has disappeared. The local system may learn about the failure only when it sends data. TCP keepalive can probe an idle peer, but operating-system defaults are often much longer than application requirements. Applications commonly define separate connection deadlines, read or idle timeouts, and protocol heartbeats.

Connection timeout
 -> establish transport
Read or idle timeout
 -> receive expected progress
Application heartbeat
 -> confirm protocol-level responsiveness

A timeout does not prove that remote application work did not occur. The server may have processed a request before the response or connection was lost. Retry safety therefore depends on application semantics, stable operation identifiers, and idempotency rather than on TCP alone.

13.7 Choosing a Transport Through Real Networks

Choose TCP when the application needs a reliable ordered stream and can tolerate waiting for missing bytes. Choose UDP when independent messages, low overhead, or application-controlled timing matters and the protocol can handle loss, duplication, ordering, and congestion responsibly. **TCP**: reliable ordered bytes, connection state, and built-in congestion control; **UDP**: independent datagrams, no handshake, and application defines recovery.

The choice is not simply “reliable versus fast.” TCP can be efficient and low latency on healthy paths, while a poorly designed UDP protocol can overload networks or spend more time recreating reliability. QUIC uses UDP as an adaptable foundation but adds secure connections, congestion control, recovery, and streams itself.

NAT and stateful firewalls treat the two transports differently because TCP exposes a visible connection lifecycle while UDP does not. For TCP: **SYN**: creates NEW state; **Handshake completes**: ESTABLISHED state; **FIN or RST**: state can close; **Idle timeout**: stale state removed. For UDP:

First permitted datagram: temporary flow state; **Matching return datagram**: permitted; **Inactivity**: state expires. A quiet UDP session may therefore need more frequent traffic to preserve mappings. TCP and UDP rules must be configured separately. **Allow TCP port 443**: does not allow UDP port 443; **Allow UDP port 443**: does not allow TCP port 443. This distinction is increasingly visible because HTTP/3 uses QUIC over UDP port 443, while HTTP/1.1 and HTTP/2 commonly use TCP port 443.

13.8 Inspecting Sockets

On Linux, `ss -ltnup` shows listening TCP and UDP sockets, while `ss -tn` shows active TCP connections. The local address reveals whether a service is bound to loopback, one interface, or a wildcard address. Socket state proves only what exists on the current host; routing, firewalls, NAT, and remote listeners still determine end-to-end reachability. Chapter 21 combines socket inspection with packet capture and evidence from the surrounding boundaries.

13.9 What to Keep in Mind

Ports identify transport endpoints inside a host, while sockets are the operating-system objects applications use. UDP sends independent datagrams without connection establishment or delivery guarantees. TCP establishes a connection, provides an ordered byte stream, retransmits missing data, protects receiver buffers through flow control, and adapts to network capacity through congestion control. Transport acknowledgments prove delivery to the remote transport stack, not business completion. Binding, timeouts, and retry safety remain application design decisions.

14. DNS and Naming

Applications normally use stable names instead of numeric addresses. The Domain Name System, or DNS, is a hierarchical distributed database that maps names to typed records. It is often described as an address book, but it also supports delegation, caching, mail routing, service metadata, verification, and security information.

Application asks for:

api.example.com

DNS returns candidate addresses:

203.0.113.25

2001:db8:100::25

DNS provides information needed before connection establishment. It does not prove that the destination is reachable, healthy, authorized, or safe.

14.1 The DNS Hierarchy and Delegation

A name is composed of labels interpreted from right to left: `api.eu.example.com`. The final dot represents the DNS root. The root delegates `.com`, `.com` delegates `example.com`, and the organization responsible for `example.com` manages its own names or delegates subdomains further. A zone is an administrative portion of this namespace served by authoritative DNS servers. Several authoritative servers normally provide redundancy. They answer from configured zone data rather than searching the Internet recursively.

`com`

-> `example.com` is served by

`ns1.example.com`

`ns2.example.com`

A circular dependency appears when the authoritative server's name lies inside the delegated zone.

To resolve `example.com`:

need address of `ns1.example.com`

To resolve `ns1.example.com`:

need `example.com` authoritative server

The parent solves this by including glue address records.

Delegation:

`example.com` -> `ns1.example.com`

Glue:

`ns1.example.com` -> 203.0.113.53

Glue lets the resolver reach the child server and then obtain authoritative data normally. Incorrect delegation or stale glue can make an otherwise correct zone unreachable from parts of the Internet.

14.2 Stub and Recursive Resolvers

Applications normally call an operating-system resolver API. The local stub resolver sends the query to one or more configured recursive resolvers supplied by DHCP, IPv6 configuration, VPN policy, or static settings. If the recursive resolver has no cached answer, it follows referrals through the root servers, the `.com` servers, and the authoritative servers for `example.com` until it obtains the requested record. The root does not return the final application address; it identifies where the next delegation can be found. The recursive resolver performs this work for the client, caches the result, and returns one final answer. A home router may forward queries to an upstream resolver. Enterprise networks commonly use internal resolvers that answer private names and forward public questions elsewhere.

14.3 Resource Records and Aliases

The client asks for both a name and a record type. Important record types include:

A

-> IPv4 address

AAAA

-> IPv6 address

CNAME

-> alias to another name

MX
-> mail exchanger
NS
-> authoritative name server
TXT
-> policy or verification text
PTR
-> reverse address-to-name mapping
SRV
-> service target and port
CAA
-> permitted certificate authorities

A name can have several A or AAAA records. The client or its connection library selects an address and may try another when the first path fails. A CNAME points to another name rather than directly to an address, allowing an organization to publish a stable name while a provider controls the target's current addresses.

www.example.com
-> web-hosting.provider.net
-> 203.0.113.80

Traditional DNS rules do not allow a CNAME to coexist with other records at the same name. This matters at a zone apex such as `example.com`, where SOA and NS records already exist. DNS providers therefore offer synthetic alias features that provide similar behavior while remaining compatible with apex requirements.

14.4 Caching, TTLs, and Negative Answers

Each record has a Time to Live, or TTL, that tells caching resolvers how long they may reuse it.

api.example.com A 203.0.113.25
TTL: 300 seconds

Caching reduces latency, traffic, and authoritative-server load, but it delays change visibility. A low TTL permits faster change at the cost of more queries; a high TTL improves efficiency but prolongs old information. Planned migrations often lower the TTL in advance, wait for existing entries to expire, change the record, and raise it again after stability is confirmed.

DNS also caches negative answers. NXDOMAIN means the queried name does not exist, while a no-data response means the name exists but lacks the requested record type.

missing.example.com A
-> NXDOMAIN
example.com AAAA
-> name exists, but no AAAA record

Creating a previously absent name or record can therefore remain invisible until the negative cache expires. Caches may exist in the application, operating system, forwarding resolver, and recursive resolver, and a long-lived transport connection may not perform DNS again at all. Clearing one layer does not clear the others.

14.5 DNS Transport and Privacy

Traditional DNS commonly uses UDP port 53. When a response is too large or marked truncated, the client can retry through TCP. Zone transfers and other larger exchanges also use TCP, so allowing only UDP can create partial failures.

DNS over TLS commonly uses port 853, while DNS over HTTPS carries queries through HTTPS, commonly on port 443. Both encrypt communication between the client and its recursive resolver. Nearby observers cannot read or modify that exchange, but the selected resolver still sees the questions and authoritative servers still receive the necessary upstream queries.

Encryption therefore does not remove the trust decision. An application that selects its own external encrypted resolver can bypass enterprise split-DNS, filtering, or local naming policy. The relevant questions are both whether the query is protected in transit and which resolver is authorized to answer it.

14.6 Internal, Split, Search, and Reverse DNS

Split DNS returns different answers depending on the client's network context.

Inside company:

`api.example.com -> 10.20.0.25`

Outside company:

`api.example.com -> 203.0.113.25`

This keeps internal traffic on private paths while exposing a public edge externally. VPN clients need the correct resolver and route to receive and use the internal view. Search domains allow a short name such as **database** to be expanded to **database.corp.example.com**, but they can create ambiguity and extra queries. Fully qualified names are clearer in application configuration. Reverse DNS uses PTR records to map an address back to a name. It is useful for diagnostics, logging, and email policy, but it is not cryptographic identity. The owner of the address space controls the reverse record.

Search domain:

`corp.example.com`

An application that asks for:

`database`

may trigger a query for: **database.corp.example.com** Several search suffixes may be tried in order. This convenience can create ambiguity and delays. A short name that fails in one suffix may cause several DNS queries before the resolver gives up. **printer:** `printer.office.example.com`, `printer.corp.example.com`, and `printer.example.com`. Fully qualified names are clearer in configuration and distributed systems. Search behavior also differs between operating systems, resolver libraries, containers, and applications. A name that resolves in an interactive shell may fail inside a container with different resolver configuration.

IPv4 addresses are reversed under `in-addr.arpa`.

`203.0.113.25`

reverse name:

`25.113.0.203.in-addr.arpa`

IPv6 reverse DNS uses hexadecimal nibbles under `ip6.arpa`. A reverse record might return:

`25.113.0.203.in-addr.arpa`

`-> PTR`

`-> server.example.com`

Reverse DNS is used for diagnostics, logging, and selected trust checks, especially in email systems. It is not guaranteed to exist, and the returned name is not automatically trustworthy. The owner of the address space controls the reverse zone. Applications should not treat a PTR value alone as authenticated identity. Forward-confirmed reverse DNS checks whether the returned name resolves back to the original address. **Address:** PTR name, A or AAAA records, and original address present?. This provides consistency, not cryptographic proof.

14.7 DNSSEC and Delegation Integrity

DNSSEC adds signatures that allow validating resolvers to verify that DNS data came from the expected zone and was not modified. A chain of trust follows signed delegations from the root toward the child zone. DNSSEC authenticates DNS data; it does not encrypt queries or authenticate the later application connection. TLS still validates the server and protects application traffic. Delegation can also require glue records when a child zone's authoritative server name lies inside that same zone. The parent supplies enough address information to reach the child server and break the circular dependency.

14.8 DNS-Based Distribution and Service Discovery

Returning several addresses provides coarse distribution and redundancy. Geographic or latency-aware DNS can return different endpoints for different resolvers, while CDNs often direct users toward an edge location. DNS stops controlling the connection after the answer is returned. Removing an unhealthy address does not immediately affect clients with cached results or established connections. Internal platforms also use DNS for service discovery. A stable service name can resolve to a virtual service address or a changing set of workload addresses. DNS tells the caller where a service might be; readiness and health mechanisms decide which instances should actually receive traffic.

14.9 Diagnosing DNS

A successful lookup proves only that a resolver returned records. If a name fails but a connection to a known address succeeds while preserving TLS SNI and HTTP authority, DNS or address selection is the likely boundary. `getent ahosts <name>` follows the operating system's normal name-service configuration, while `dig` exposes record types, TTLs, and resolver-specific answers.

Different users can receive different results because of normal caching, split DNS, geographic policy, negative caching, or one unhealthy address in a returned set. The useful questions are which resolver answered, which record type was requested, which addresses were returned, and whether the application reused an older cached result or an existing connection.

Failure patterns then become easier to classify. A name failing everywhere suggests missing records, broken delegation, unavailable authoritative servers, or DNSSEC validation failure. Failure only on one network suggests split-DNS, VPN, search-domain, or resolver configuration. IPv4 succeeding while IPv6 fails usually means DNS successfully returned an `AAAA` record but the selected IPv6 route, firewall, listener, or server path is broken. Chapter 21 places these observations inside the full route, transport, TLS, and application workflow.

14.10 What to Keep in Mind

DNS is a delegated hierarchical database. Applications ask a stub resolver, recursive resolvers follow referrals and cache answers, and authoritative servers publish zone data. `A` and `AAAA` records contain addresses, while aliases and other types describe additional services and policy. TTLs make DNS efficient but delay change visibility. Split DNS and service discovery can return context-dependent endpoints. DNS supplies candidates for connection establishment; routing, transport, TLS, and application health remain separate stages.

15. TLS, Certificates, and Secure Connections

IP and transport protocols can deliver data across untrusted networks, but they do not prevent observers from reading or changing it and do not prove that the remote endpoint is the intended service. Transport Layer Security, or TLS, adds confidentiality, integrity, and peer authentication.

HTTP/1.1 or HTTP/2

-> TLS

-> TCP

HTTP/3

-> TLS integrated into QUIC

-> UDP

TLS protects a connection. It does not authenticate the application user, authorize business operations, or guarantee that the server application is free from vulnerabilities.

15.1 Symmetric and Public-Key Cryptography

Symmetric encryption uses one shared secret for encryption and decryption and is efficient enough for bulk traffic. The challenge is establishing that secret safely. Public-key cryptography uses a private key and a related public key. A server signs handshake data with its private key, and the client verifies the signature with the public key. This proves control of the private key but does not by itself prove that the key belongs to `api.example.com`. TLS uses public-key techniques during the handshake to authenticate the peer and derive shared secrets. Application data is then protected with efficient symmetric authenticated encryption. **Handshake:** authenticate peer and derive session keys; **Application traffic:** symmetric encryption and integrity.

15.2 Certificates, Trust Chains, and Issuance

A certificate binds a public key to names and usage constraints. Modern HTTPS validation relies primarily on the Subject Alternative Name extension.

Valid DNS names:

`api.example.com`

`www.example.com`

A wildcard such as `*.example.com` normally covers one label, such as `api.example.com`, but not the apex `example.com` or a deeper name such as `v2.api.example.com`. Certificate authorities validate control of a name and sign certificates. Operating systems and browsers contain trust stores of root CA certificates. A typical chain is: **Trusted root CA:** intermediate CA and `api.example.com` certificate. The server sends its leaf certificate and required intermediates. The client already trusts the root independently. Sending an untrusted root from the server does not make it trusted.

Public certificate issuance is commonly automated through protocols such as ACME. Private certificate authorities can issue internal service and device identities, but clients must explicitly trust the private root.

Public certificate authorities commonly validate that the requester controls the domain. Validation may use: **DNS challenge:** publish required DNS record; **HTTP challenge:** serve required token at specific URL; **TLS challenge:** respond with special certificate or handshake. Automated protocols such as ACME allow servers and certificate-management systems to request and renew certificates without manual intervention. **Server:** requests certificate; **CA:** issues challenge; **Server:** proves domain control; **CA:** issues certificate. Domain validation confirms control of the name, not legal identity or application quality. Organizations can also operate private certificate authorities for internal services. Clients must trust the private root certificate explicitly.

An organization can create its own certificate authority for internal services.

Company root CA: company intermediate CA and internal server certificates. Clients must trust the company root certificate. **Managed workstation:** root installed through device management; **Unmanaged device:** root not trusted and certificate warning. Private PKI allows certificates for internal names and devices that public CAs should not issue. The root private key is highly sensitive. If an attacker obtains it, they can create certificates trusted throughout the organization. Root keys are therefore often stored offline or in hardware security modules, while intermediates perform routine issuance. Trusting a private root grants it authority over every name and usage allowed by the client trust system, not only one internal domain. Deployment should therefore be tightly controlled.

15.3 Certificate Validation

The client validates several independent conditions:

- Signature chain reaches a trusted root
- Requested hostname appears in certificate
- Current time is within validity period
- Certificate permits server authentication
- Applicable policy and revocation checks pass

The hostname is the name the client intended to reach, not merely the IP address returned by DNS. This allows many names to share one address while retaining separate identities. Connecting directly to an IP address often fails when the certificate contains only DNS names. A missing intermediate can work on one client that cached it and fail on another that did not, creating confusing inconsistency. A valid certificate means that a trusted authority approved the binding between the name and public key. It does not mean the website is honest or that the user is authorized to perform an operation.

15.4 The TLS Handshake

In a simplified TLS 1.3 handshake, the client sends a ClientHello containing supported versions, cryptographic parameters, an ephemeral key contribution, the intended server name, and application protocols. The server selects compatible parameters, returns its own key contribution and certificate chain, and proves control of the certificate's private key.

```
ClientHello
-> versions, key share, SNI, ALPN
ServerHello and certificate
-> selected parameters and identity
Finished messages
-> verify handshake integrity
Encrypted application data
```

Both sides derive fresh symmetric keys. Modern ephemeral Diffie-Hellman exchange provides forward secrecy: stealing the server's long-term private key later does not by itself reveal previously recorded sessions. The handshake also verifies that an attacker did not silently change the negotiated parameters.

15.5 SNI and ALPN

One IP address can host several secure services. Server Name Indication, or SNI, carries the intended hostname in the handshake so the server can choose the correct certificate before it receives encrypted HTTP data.

```
SNI:
  api.example.com
```

Traditional SNI is visible to network observers. Encrypted ClientHello can protect more of this metadata when the client, DNS configuration, and server support it. Application-Layer Protocol Negotiation, or ALPN, selects the protocol used inside TLS. A client can offer `h2` and `http/1.1`, and the server chooses one. QUIC integrates TLS 1.3 and uses ALPN to select HTTP/3 or another QUIC application protocol.

15.6 Records, Resumption, and 0-RTT

After the handshake, TLS divides application bytes into authenticated encrypted records. Record boundaries are independent of TCP segments and application messages; one TLS record can span several TCP segments, and one segment can contain parts of several records. Session resumption reduces the cost of later connections. The server issues resumption information, commonly a session ticket, and the client uses it in an abbreviated future handshake. New session keys are still derived. TLS 1.3 can permit early 0-RTT data during resumption, but that data has weaker replay protection. It should be used only for operations safe to repeat, such as selected reads—not payments or state-changing requests without idempotency protection.

15.7 Mutual TLS and Service Identity

Ordinary HTTPS authenticates the server to the client. Mutual TLS also requests a client certificate and allows the server to validate the caller's cryptographic identity. **Server certificate:** client validates service; **Client certificate:** server validates client or workload. mTLS is common for service-to-service communication, managed devices, and industrial systems. It authenticates the connection identity; application authorization

must still decide which operations that identity may perform. Certificate issuance, rotation, revocation, and private-key protection become operational responsibilities. Short-lived automatically renewed certificates reduce exposure but require reliable automation and monitoring.

15.8 TLS Termination and Inspection

TLS may terminate at a CDN, reverse proxy, load balancer, or API gateway rather than inside the final application process.

Client

-> TLS connection 1

Edge proxy

-> HTTP, TLS, or mTLS connection 2

Application

The proxy sees plaintext and becomes a security boundary. The backend sees the proxy as its network peer, so the edge must replace untrusted forwarding headers before adding trustworthy client address, host, and scheme information. The internal hop can use plain HTTP inside a deliberately controlled boundary, HTTPS, or mTLS according to risk and identity requirements.

Enterprise TLS inspection uses the same two-connection model with a private CA installed on managed clients. It permits policy and malware inspection but gives the intermediary decryption capability and private signing authority. Applications using mutual TLS, pinning, unusual protocols, or hardware-backed trust may reject interception. TLS inspection therefore changes the trust model rather than merely observing an otherwise end-to-end connection.

15.9 Certificate Lifecycle and Failure Patterns

Certificates expire, private keys rotate, and trust stores differ. Reliable operation therefore needs automated issuance, deployment, renewal, reload, and monitoring of the certificate actually presented by every endpoint. A safe rotation overlaps old and new validity: deploy the new certificate, verify all load-balanced nodes use it, allow existing connections to finish, and then retire the old one.

Common deployment failures

- > certificate renewed but service not reloaded
- > one node still presents the old certificate
- > intermediate chain missing
- > certificate and private key do not match
- > client or server clock is wrong

A certificate may need to be distrusted before expiration after key compromise or incorrect issuance. CRLs, OCSP, browser-maintained revocation information, and short certificate lifetimes provide imperfect but complementary responses. Certificate Transparency makes publicly trusted issuance observable. Custom certificate pinning narrows accepted identities but can cause outages during legitimate key rotation and should be reserved for tightly controlled deployments.

Failure location matters. A name mismatch usually indicates the wrong virtual host, missing hostname, connection by raw address, or incorrect SNI routing. An untrusted chain suggests a missing intermediate, unavailable private root, or obsolete trust store. Intermittent failure behind a load balancer often indicates inconsistent certificates or clocks between nodes. `openssl s_client` reveals the certificate and negotiated TLS parameters; Chapter 21 combines that evidence with DNS, transport, HTTP, and backend observations.

15.10 What to Keep in Mind

TLS authenticates the server connection, encrypts application traffic, and detects modification. Public-key cryptography and certificates establish identity during the handshake, while symmetric keys protect bulk data efficiently. Clients validate the chain, hostname, validity period, and permitted usage. SNI selects a hosted TLS identity, ALPN selects the application protocol, resumption reduces later setup cost, and mTLS can authenticate clients or workloads. TLS ends wherever it is terminated, so proxies and internal hops create additional trust boundaries.

16. HTTP Requests, Responses, and Semantics

TLS creates a secure channel, but it does not define what an application asks for or how the server reports the result. The Hypertext Transfer Protocol, or HTTP, provides that structure. Its core model is a client request followed by a server response, even though modern implementations may multiplex many exchanges over one connection. **Request:** method, target, headers, optional body; **Response:** status, headers, optional body.

HTTP began as a protocol for hypertext documents and now carries websites, APIs, media metadata, software updates, telemetry, and many other representations. The wire encoding differs between HTTP/1.1, HTTP/2, and HTTP/3, but the semantics remain broadly consistent.

16.1 URLs, Targets, Headers, and Bodies

Consider:

```
https://api.example.com:8443/orders/42?include=items#summary
```

The scheme is `https`, the host is `api.example.com`, the port is `8443`, the path is `/orders/42`, and the query is `include=items`. The fragment `summary` is interpreted by the client and is not normally sent to the server. DNS resolves the host, routing reaches the selected address, and TLS authenticates the host name. HTTP then sends the path, query, authority, and operation through the established connection. Headers carry metadata. Request headers can describe accepted response formats, credentials, body type, cache conditions, and client preferences. Response headers can describe the returned representation, caching policy, cookies, redirects, and security rules.

```
Host: api.example.com
Accept: application/json
Authorization: Bearer ...
Content-Type: application/json
```

HTTP/1.1 uses the `Host` header because several websites can share one address. HTTP/2 and HTTP/3 represent the same concept with `:authority`. It is related to TLS SNI but used at a later layer: SNI selects the secure endpoint and certificate, while HTTP authority selects the application virtual host or route. A request or response can include a body. `Content-Type` describes its representation, such as JSON, plain text, form data, or arbitrary bytes. Servers should enforce body-size limits and validate both syntax and business meaning. Large bodies can be streamed rather than buffered, but streaming does not remove the need for deadlines, flow control, and resource limits.

HTTP/1.1 uses `Content-Length`, chunked transfer encoding, or connection closure to frame bodies. HTTP/2 and HTTP/3 use binary frames and explicit stream completion. Intermediaries must agree on framing; inconsistent parsing of ambiguous lengths can create request-smuggling vulnerabilities.

16.2 Methods, Safety, and Idempotency

The method describes the intended operation:

```
GET
-> retrieve a representation
HEAD
-> retrieve metadata without the body
POST
-> submit data for processing or create subordinate state
PUT
-> create or replace a resource at a known target
PATCH
-> apply a partial change
DELETE
-> remove a resource
OPTIONS
-> discover communication options
```

A safe method is intended not to change the resource state requested by the user. `GET`, `HEAD`, and `OPTIONS` are conventionally safe, although they can still produce logs, metrics, and cache updates. Destructive actions should not be exposed through `GET`, because browsers, crawlers, and scanners may follow links automatically. An idempotent operation has the same intended effect when repeated. `PUT` that replaces one known resource is

normally idempotent, and repeating a **DELETE** should leave the resource absent. **POST** is not generally idempotent: retrying an order-creation request can create another order.

Retry safety depends on these semantics. A non-idempotent operation can use a stable resource identifier or idempotency key so the server recognizes repeated transport attempts as one intended business operation.

POST /payments

Idempotency-Key: payment-7f31

The server stores the result for that key and returns it again rather than executing the payment twice.

16.3 Status Codes and Redirects

A response status code describes the HTTP outcome. The first digit identifies a broad class:

```
1xx informational
2xx success
3xx redirection
4xx request or client problem
5xx server or upstream failure
```

Common successful results include **200 OK**, **201 Created**, **202 Accepted**, and **204 No Content**. **201** often includes a **Location** header for the new resource. **202** means work was accepted but has not necessarily completed. Important client-error statuses include:

```
400 malformed or invalid request
401 authentication required or failed
403 request understood but forbidden
404 resource not found or intentionally hidden
405 method unsupported for this target
409 state conflict
412 precondition failed
413 body too large
415 media type unsupported
422 syntactically valid but semantically invalid content
429 rate limit exceeded
```

Despite its name, **401 Unauthorized** concerns authentication; **403 Forbidden** is the more direct authorization denial. **500 Internal Server Error** represents an unexpected server failure. **502 Bad Gateway** means a proxy could not obtain a valid upstream response, **503 Service Unavailable** indicates temporary inability to serve, and **504 Gateway Timeout** means an intermediary waited too long for its upstream. A **502** or **504** proves that the client reached a gateway and narrows the problem to the next part of the path.

Redirects use a **Location** header. **307** and **308** preserve the method and body, while **303 See Other** tells the client to retrieve another resource, normally with **GET**. Historical behavior around **301** and **302** can change **POST** to **GET**, so APIs should choose redirect codes deliberately.

16.4 Representations and Content Negotiation

One resource can have several representations. A client can send:

```
Accept: application/json
Accept-Language: hr-HR, en;q=0.7
Accept-Encoding: br, gzip
```

The server reports what it selected through headers such as **Content-Type** and **Content-Encoding**. Compression saves bandwidth for text and structured data but costs CPU and provides little benefit for formats already compressed. When a response varies according to request headers, **Vary** tells shared caches which values belong in the cache key. **Vary: Accept-Encoding, Accept-Language** Excessive variation fragments the cache and reduces reuse. Public APIs often prefer one explicit representation rather than broad negotiation.

16.5 Conditional Requests, Concurrency, and Ranges

HTTP validators allow clients and caches to avoid transferring unchanged representations. An entity tag identifies one version: **Etag: "order-42-v18"** A later request can send **If-None-Match**. If the representation is unchanged, the server returns **304 Not Modified** without the full body. For updates, **If-Match** implements

optimistic concurrency: the server changes the resource only if the client's version is still current, otherwise it returns **412 Precondition Failed**. Date-based validators use **Last-Modified** and conditional date headers, but ETags can describe application versions more precisely.

Range requests ask for selected bytes and receive **206 Partial Content**. They support resumed downloads and media seeking. Servers and caches must apply ranges to the correct representation, especially when compression changes the byte sequence.

A client can request part of a representation. **Range: bytes=1000000-1999999** A successful partial response uses: **206 Partial Content** with: **Content-Range: bytes 1000000-1999999/5000000** Range requests support:

Resuming downloads

Seeking in media

Fetching selected file sections

The server can return **416 Range Not Satisfiable** when the range is invalid. Range support interacts with compression and caching, so servers and CDNs must handle representation identity carefully.

16.6 Cookies, Authentication, and Browser Origins

A server can ask a browser to store a cookie:

Set-Cookie: session=abc123; Path=/; Secure; HttpOnly; SameSite=Lax

Applicable cookies are sent on later requests. **Secure** limits transmission to secure connections, **HttpOnly** keeps the value away from ordinary browser scripts, **SameSite** controls selected cross-site use, and domain and path attributes define scope. Cookies carry application state across independent requests; they are not transport connection identifiers. A session can survive several TCP or QUIC connections, while one pooled connection can carry many requests.

HTTP also defines authentication challenges and authorization headers. Basic authentication only encodes credentials and must be protected by TLS. Bearer tokens grant authority to whoever possesses them and should not appear in URLs, logs, or unprotected storage. Applications can also use cookies, signed requests, mutual TLS, and protocol-specific credentials.

A browser origin is the combination of scheme, host, and port.

https://example.com

http://example.com

https://api.example.com

https://example.com:8443

These are four different origins. The same-origin policy prevents scripts from reading arbitrary cross-origin responses. Cross-Origin Resource Sharing, or CORS, lets a server grant selected browser origins access through response headers and, for some requests, a preflight exchange.

OPTIONS /orders

Origin: https://app.example.com

Access-Control-Request-Method: POST

CORS is browser enforcement, not a network firewall. Command-line tools and other servers can still send the request, so every operation still requires appropriate authentication and authorization.

16.7 Proxies, Forwarded Context, and Stateless Semantics

A reverse proxy creates a new upstream connection, so the application sees the proxy as its direct network peer. Trusted forwarding headers can preserve the original client address, scheme, and host.

Forwarded

X-Forwarded-For

X-Forwarded-Proto

X-Forwarded-Host

The application must trust these fields only from approved proxies that remove or replace untrusted values. Otherwise a client can spoof audit records, generated URLs, rate-limit identity, or the apparent security of the request. HTTP is stateless at the protocol level: each request can be understood independently. Applications build state through cookies, tokens, database records, workflow identifiers, and resource versions. Connection

reuse does not imply that one process must remember the previous request, which makes horizontal scaling easier.

16.8 Deadlines, Ambiguous Outcomes, and Inspection

An HTTP operation includes name resolution, connection establishment, TLS, body upload, server processing, response headers, and body download. Different limits can apply to each stage. A client timeout does not prove that the server performed no work; it may have committed an operation before the response was lost. Reliable APIs use idempotent methods, stable resource identifiers, idempotency keys, or status queries when repeating an ambiguous operation could create a duplicate effect.

`curl -v` can expose resolution, transport, TLS, request headers, redirects, and response status in one test. Browser developer tools add cache, cookie, CORS, and resource-waterfall evidence. Chapter 21 places those observations inside the complete troubleshooting path.

16.9 What to Keep in Mind

HTTP defines application requests and responses above the transport. Methods describe intended operations; safe and idempotent semantics influence caching and retry behavior. Headers carry metadata, bodies carry representations, and status codes localize the outcome. Validators support cache revalidation and optimistic concurrency, while cookies and authorization headers carry application identity or session state. Proxies create new connection boundaries and must establish trustworthy forwarding context. A timeout leaves the remote business result uncertain, so reliable APIs make retry behavior explicit.

17. HTTP Connections and Delivery Infrastructure

A production HTTP request often passes through a CDN, reverse proxy, load balancer, and one or more application services. Each intermediary can improve performance, security, and availability, but it also introduces another connection, timeout, cache, routing rule, and failure boundary.

Client

- > CDN or edge proxy
- > regional load balancer
- > application service
- > internal dependency

17.1 Connection Reuse and HTTP Versions

Creating a new TCP and TLS connection for every request repeats handshakes, consumes ephemeral ports, and begins congestion control cautiously. HTTP/1.1 therefore uses persistent connections, and clients maintain connection pools by destination. Reusing a configured client allows the pool to control concurrent connections, idle time, maximum lifetime, and DNS refresh. Constructing and discarding a client for every request can defeat pooling.

HTTP/1.1 normally processes responses in request order on one connection. Browsers historically used several parallel connections because one slow response could delay later work.

HTTP/2 preserves HTTP semantics but encodes them into binary frames on independent streams inside one TCP connection.

One HTTP/2 connection

- > stream 1: HTML
- > stream 3: CSS
- > stream 5: API request

Frames from several streams can be interleaved, so one slow application response does not block another merely because it was requested first. All streams still share one ordered TCP byte stream. When a TCP segment is lost, later bytes for every stream wait until the gap is repaired.

HTTP/3 carries HTTP through QUIC over UDP. QUIC integrates TLS 1.3, loss recovery, congestion control, connection identifiers, and multiple reliable streams. Loss in one stream does not block unrelated streams. A connection identifier can also let a mobile client validate and continue over a new network path. UDP port 443 must still pass through firewalls and NAT, and clients fall back to HTTP/2 or HTTP/1.1 when QUIC is unavailable.

17.2 Proxies and Connection Boundaries

A forward proxy represents clients when they access external systems. For HTTPS, the client commonly sends `CONNECT host:443` and creates TLS through the resulting tunnel. Unless the proxy performs TLS inspection, it sees connection metadata but not encrypted HTTP contents.

A reverse proxy represents servers. It can terminate TLS, validate hostnames, enforce authentication and rate limits, normalize headers, compress responses, cache content, and select backends. The client-facing and backend-facing connections can use different HTTP versions and security policies.

Client

- > connection A

Reverse proxy

- > connection B

Backend

API gateways, ingress controllers, CDNs, service-mesh proxies, and managed HTTP load balancers are variations of this pattern. The application request can span several transport connections, and every termination point becomes a new timeout, trust, and observability boundary.

Proxies also decide whether to buffer. Buffering a slow upload before opening the backend connection protects application capacity, while buffering a response can isolate a fast backend from a slow client. The same behavior can break incremental streaming or increase disk and memory use, so streaming endpoints need explicit proxy configuration. Compression reduces transfer size for text formats but consumes CPU and requires caches to distinguish encoding variants through `Vary: Accept-Encoding`.

17.3 Caching and Distributed Delivery

An HTTP cache stores reusable responses and returns them while they remain fresh. Caches exist in browsers, client libraries, proxies, gateways, and CDNs. Their keys normally include the scheme, authority, path, query, and request headers named by **Vary**.

Cache-Control defines policy. **max-age** gives a freshness lifetime, **s-maxage** can specify a shared-cache lifetime, **public** permits shared reuse, **private** restricts it to a user-specific cache, **no-store** forbids storage, and **no-cache** permits storage but requires validation before reuse.

Revalidation uses ETags or modification dates. A **304 Not Modified** refreshes the cached response without transferring the body. Static assets are easiest to cache when a content hash appears in the filename; a deployment publishes a new name rather than trying to invalidate every old copy.

Cache correctness is part of security. A shared cache must not return one user's response to another, and attacker-controlled input must be included in the cache key when it changes the representation. Host validation, URL normalization, **Vary**, and private/no-store policy are therefore correctness boundaries.

Changing the origin does not instantly remove distributed copies. Applications use TTLs, versioned URLs, conditional requests, purge APIs, or surrogate keys according to how much staleness the data can tolerate. Selected caches can serve stale public content during origin failure, but permissions, balances, and other correctness-sensitive state require stricter freshness.

A Content Delivery Network places cache and proxy capacity near users. DNS, anycast routing, or both select an edge site. The edge can answer locally or fetch from the origin, reducing latency and origin load while also absorbing traffic spikes. With anycast, several sites advertise the same prefix and routing selects a preferred available location. The selected site follows network policy rather than simple physical distance.

17.4 Load Distribution and Failure Boundaries

A load balancer presents one endpoint and distributes traffic across several backends. A layer-4 balancer chooses using transport information, while a layer-7 balancer understands HTTP and can route by host, path, method, or header. Round-robin selection spreads requests evenly when backends are similar, least-connections favors less occupied instances, and weighted policies account for different capacities. Hash-based affinity can keep related traffic together, but every algorithm sees only the signals available at that boundary.

Health checks determine which targets are eligible. A transport check proves that a port accepts connections. An HTTP readiness check can verify that initialization is complete and the service is prepared for new work. Liveness asks whether a process should be restarted; readiness asks whether it should receive traffic. Checking every remote dependency inside readiness can remove every application instance during one shared dependency slowdown.

During deployment, the backend should become unready before termination and allow active requests or streams to drain. Immediate termination creates resets and retries while capacity is already changing. Sticky sessions keep one client on one backend but concentrate load and lose state when that backend fails. Shared state or signed client state normally allows any healthy instance to continue.

A proxy may retry a failed upstream request on another backend. That is safe only when the operation is idempotent, the failure proves processing did not begin, or the application uses an idempotency key. Independent retries in the client, CDN, proxy, service mesh, and application can multiply traffic during an outage.

Timeouts also nest. Database and internal service limits should normally expire before the application, proxy, and client deadlines above them. Otherwise a proxy can return **504** while the application continues and commits work the client believes failed.

17.5 Long-Lived Streams and Reconnection

Not every HTTP interaction ends after one small response. Applications can stream large bodies, use Server-Sent Events for one-way updates, upgrade to WebSockets for bidirectional messages, or use streaming RPC frameworks. These patterns keep transport and proxy state alive for much longer than an ordinary request.

A transport connection is not the same as an application session or durable state.

Connection

-> temporary path

Application session

-> authenticated continuity

Durable cursor

-> what the client has already processed

Mobile networks change, proxies enforce idle limits, deployments drain connections, and machines restart. A reconnecting client may need to authenticate again, provide the last received sequence or cursor, replay missed events, and ignore duplicates. Heartbeats can confirm protocol-level responsiveness and preserve intermediary state, but they should not be so frequent that they waste capacity or battery. Backpressure also remains necessary: a streaming producer must not continue filling memory when the client or network consumes data more slowly. HTTP/2 and QUIC flow control help at the protocol level, while the application still needs bounded buffers and cancellation.

Buffering must be chosen deliberately. A proxy that buffers a slow upload protects the application from holding an upstream connection, while buffering a streaming response can prevent the client from receiving incremental data. Every long-lived protocol needs documented idle timeouts, maximum duration, cancellation, and resume behavior.

17.6 Global Distribution and Failure Localization

A global service can combine several selection layers:

DNS

-> endpoint or region

Anycast

-> edge location

CDN

-> cache or origin path

Regional load balancer

-> healthy application instance

Each layer solves a different problem. DNS changes where new clients connect, anycast changes which edge receives packets, a CDN can serve without reaching the origin, and a load balancer removes unhealthy instances. The trade-off is a path containing several connection pools, caches, deadlines, and policy boundaries.

Infrastructure status codes localize failure. 502 means the client reached a proxy that could not obtain a valid upstream response. 504 means the proxy waited beyond its upstream deadline. If application logs contain no matching request, inspect the proxy-to-backend route, DNS, TLS, listener, and firewall. If the application completed after the proxy returned 504, timeout and cancellation layers are misaligned rather than the network being simply down.

17.7 What to Keep in Mind

Persistent connections avoid repeated transport and TLS setup. HTTP/2 multiplexes streams over TCP, while HTTP/3 uses QUIC to avoid cross-stream blocking and support path migration. Proxies terminate one connection and create another, establishing new trust, timeout, and observability boundaries. Caches and CDNs trade freshness for lower latency and origin load, while load balancers select ready backends. Long-lived streams need explicit idle, shutdown, reconnection, and resume behavior. Retries, buffering, timeout ordering, and cache policy must follow application semantics because infrastructure cannot infer whether duplicate work or stale data is safe.

18. From a URL to a Server Response

The previous chapters introduced each mechanism separately. We can now follow one request from a laptop to a public service and back. Assume the laptop is connected to home Wi-Fi and the user opens `https://shop.example.com/products/42`.

The browser experiences one logical action. The network implements it as a sequence of decisions made at different scopes and often through several separate connections.

18.1 Existing Local State

Before the request begins, the laptop has associated with an access point and obtained configuration. A possible IPv4 state is:

```
Address: 192.168.1.42/24
Gateway: 192.168.1.1
DNS:     192.168.1.1
```

The operating system has a connected route for `192.168.1.0/24` and a default route through the gateway. Its neighbor table may already contain the router's MAC address. The browser and operating system may also have cached DNS records, reusable connections, TLS resumption information, cookies, and HTTP responses. We will follow the full path as if no reusable state exists.

The browser parses the URL into a scheme, authority, and path. `https` selects secure HTTP and default port 443; `shop.example.com` is the name whose service identity must be resolved and authenticated; `/products/42` becomes the HTTP target.

18.2 DNS Resolution and Address Selection

The browser or operating system first checks local caches and then asks its configured recursive resolver for `A` and `AAAA` records. The resolver can answer from cache or follow referrals through the DNS hierarchy until an authoritative server supplies addresses.

```
shop.example.com
-> AAAA 2001:db8:100::25
-> A    203.0.113.25
```

The client chooses an address family according to platform policy and connection progress. It may start with IPv6 and try IPv4 shortly afterward if the first path does not advance. Assume this network selects IPv4 address `203.0.113.25`.

DNS has completed only naming. The client still needs a route, transport connection, secure identity, and working application.

18.3 Route, Source Endpoint, and Local Frame

The operating system asks its routing table how to reach `203.0.113.25`. No local prefix matches, so the default route selects gateway `192.168.1.1` and the Wi-Fi interface. The browser requests a TCP connection to port 443, and the operating system selects an ephemeral source port.

```
Local endpoint:
  192.168.1.42:51500
Remote endpoint:
  203.0.113.25:443
```

The packet's IP destination is the public server, but the first local frame is addressed to the router. If the router's MAC address is not cached, ARP discovers it.

```
Wi-Fi frame destination:
  router MAC
IP packet destination:
  203.0.113.25
TCP destination port:
  443
```

The access point receives the wireless frame and bridges the packet into the local network. Consumer equipment may combine the access point, switch, router, firewall, and NAT functions, but the packet still crosses those logical responsibilities.

18.4 TCP, NAT, and Internet Routing

The client sends a TCP SYN. The home router permits the outbound connection and creates a NAT mapping.

```
Private flow:
192.168.1.42:51500
  -> 203.0.113.25:443
Public representation:
198.51.100.20:62001
  -> 203.0.113.25:443
```

The router creates a new provider-facing frame and forwards the translated packet. Each later router removes the local frame, performs longest-prefix matching on the IP destination, decrements the TTL, and creates another frame for the next link.

```
Receive frame
  -> inspect IP destination
  -> select route
  -> decrement hop limit
  -> create next-link frame
```

The Internet path emerges from local routing decisions. The packet does not carry a complete route list, and the return path need not use the same routers.

The destination network eventually delivers the SYN to a public load balancer, reverse proxy, or server. A listening socket on port 443 accepts the connection and returns SYN-ACK. The home router recognizes the reply as established traffic, reverses the NAT mapping, and sends a local frame to the laptop. The final ACK completes the TCP handshake.

18.5 TLS and HTTP

The browser sends a TLS ClientHello containing supported versions, an ephemeral key contribution, SNI for `shop.example.com`, and ALPN choices such as `h2` and `http/1.1`.

The server or edge proxy returns a certificate chain and proves control of the private key. The browser verifies the hostname, validity period, permitted use, signature chain, and handshake proof. Both sides derive symmetric keys, and ALPN selects HTTP/2.

The browser then sends the semantic equivalent of:

```
GET /products/42
Host: shop.example.com
Accept: application/json
Cookie: ...
```

HTTP/2 encodes the request into binary frames on one stream. TLS protects those bytes, TCP divides them into segments, IP places them into packets, and each local link carries the packets inside its own frames.

18.6 Edge, Backend, and Application

A reverse proxy terminates the client TLS connection and reconstructs the HTTP request. It can validate the authority, enforce size and rate limits, authenticate the caller, add trusted forwarding context, and look for a cached response.

If it needs the origin, the proxy selects a ready backend and creates or reuses another transport connection. The backend operating system delivers the request to a listening socket, and the web framework matches `/products/42` to an application route. The application may call a database or another service, each of which creates another nested communication path.

The application produces a response such as:

```
200 OK
Content-Type: application/json
```

Cache-Control: public, max-age=60
ETag: "product-42-v18"

The proxy can compress it, add security or cache headers, record timing, and return it through the existing client connection.

18.7 Returning, Reusing, and Caching

The response passes through HTTP/2 frames, TLS records, TCP segments, IP packets, routed links, reverse NAT, and local Wi-Fi frames. TCP acknowledges received byte ranges and retransmits loss. The browser decrypts TLS records, reconstructs the HTTP stream, and processes the representation.

A later request can be much shorter. The browser may reuse the HTTP/2 connection, open another stream, resume TLS after reconnecting, reuse a DNS answer, or return a fresh cached response without contacting the network. After the response becomes stale, `If-None-Match` can revalidate the ETag and receive `304 Not Modified`.

The return path across the Internet may differ from the forward path. That is harmless while both directions remain deliverable and every stateful intermediary sees the traffic it requires.

18.8 How Native IPv6 Changes the Journey

With native IPv6, the structure remains the same but selected mechanisms change.

IPv4 home path

- > private address
- > ARP for gateway
- > commonly source NAT

Native IPv6 path

- > global IPv6 address
- > Neighbor Discovery for gateway
- > normally no address-sharing NAT
- > stateful firewall still applies

DNS supplies an AAAA address, the route selects an IPv6 gateway, Neighbor Discovery resolves the local next hop, and routers forward the packet using IPv6 prefixes and Hop Limit. The application still uses a transport endpoint, TLS still authenticates `shop.example.com`, HTTP still describes the request, and proxies can still terminate and recreate connections.

18.9 Identifiers and Failure Boundaries

The same request uses different identifiers at different scopes.

Application:

`https://shop.example.com/products/42`

DNS:

`shop.example.com` -> address

Transport:

client address:port -> server address:443

Local frame:

laptop MAC -> router MAC

Application identity:

cookie or token

A user-visible failure can occur at Wi-Fi association, address configuration, route selection, neighbor discovery, DNS, NAT or firewall state, TCP or QUIC, TLS, HTTP infrastructure, application code, or a downstream dependency. The next chapters extend the path through VPN and virtualized environments before Chapter 21 turns the model into one systematic diagnostic method.

18.10 What to Keep in Mind

A URL request is not one indivisible packet. DNS resolves a service name, the operating system selects an address, route, source endpoint, and next hop, local frames carry packets one link at a time, routers forward them, and NAT may rewrite an IPv4 client endpoint. TCP or QUIC creates transport, TLS authenticates and protects it, HTTP describes the operation, proxies select backends, and the application creates the response.

The logical request can cross many frames and several transport connections while still appearing to the user as one action.

Part V — Applied Networking and Diagnosis

19. VPNs, Tunnels, and Private Network Overlays

A tunnel places one packet inside another so it can cross a network that does not directly route or understand the original communication. Tunnels are used by remote-access VPNs, site-to-site links, cloud networks, container overlays, and mobility systems. Encapsulation creates a logical path; encryption, integrity, and peer authentication are separate properties added by secure VPN protocols.

19.1 Inner and Outer Packets

Suppose two offices use private networks:

Office A: 10.10.0.0/16

Office B: 10.20.0.0/16

A gateway in Office A can take an inner packet from 10.10.0.25 to 10.20.0.50 and place it inside an outer packet sent between public tunnel endpoints.

Inner packet

10.10.0.25 -> 10.20.0.50

Outer packet

198.51.100.10 -> 203.0.113.20

Internet routers inspect the outer destination. The remote gateway removes the outer header and routes the inner packet into Office B.

Operating systems commonly represent a tunnel as a virtual interface such as `tun0` or `wg0`. A route to 10.20.0.0/16 selects that interface. The tunnel then performs another route lookup for the public peer through the physical network.

Inner route

-> protected prefix through tunnel

Outer route

-> tunnel peer through underlay

The outer endpoint must remain reachable without using the tunnel itself, or recursive routing prevents the tunnel from being established.

19.2 Tunnel Overhead and Secure VPN Properties

Encapsulation adds outer IP, transport, tunnel, and cryptographic overhead. A packet that fits a 1500-byte local link can become too large after those headers are added.

Physical MTU:

1500 bytes

Tunnel overhead:

60 bytes

Safe inner packet:

about 1440 bytes

Tunnel interfaces therefore commonly use a smaller MTU. Path MTU Discovery and TCP MSS adjustment can also prevent oversized packets. A characteristic failure is that the tunnel connects and small requests work while large uploads or responses stall because required ICMP Packet Too Big information is blocked.

Tunneling alone does not provide security. A VPN normally adds encryption, integrity, replay protection, and peer authentication. Peers can authenticate through public keys, certificates, pre-shared keys, user credentials, device identity, or a combination. Authentication should lead to authorization: a connected developer may need selected source-control and build services without receiving routes to every database and management subnet. A VPN creates a protected path, not automatic trust in every packet or user.

19.3 Remote Access and Split Tunneling

A remote-access VPN connects one device to a private environment. The client receives a virtual address, routes, and often internal DNS settings.

Company prefixes
-> VPN tunnel
Public Internet
-> local gateway

This is split tunneling. A full tunnel sends nearly all traffic through the VPN gateway, centralizing egress policy and logging but adding latency and gateway load.

The private network needs a return route toward the VPN client pool. A gateway can instead source-NAT remote clients, which simplifies return routing but hides individual client addresses from internal systems.

A kill-switch policy can block protected traffic when the tunnel disappears, while still permitting the outer connection needed to reach the VPN gateway. Always-on and on-demand modes differ mainly in when that protected path is established; both require reliable recovery as the device changes networks.

19.4 Site-to-Site VPNs and Routing

A site-to-site VPN connects networks rather than one user device. Hosts send remote private prefixes toward a local gateway, and the gateways handle encapsulation and security.

Office A
<-> encrypted tunnel
Office B

A hub-and-spoke topology simplifies central policy but can detour branch-to-branch traffic. A full mesh creates more direct paths but more tunnel relationships.

Route-based VPNs expose logical interfaces and integrate naturally with static or dynamic routing. Policy-based VPNs protect traffic matching configured source and destination selectors. A mismatch between peers can make only one prefix or direction work. Route-based designs usually scale more cleanly when many prefixes, backup paths, or routing protocols are involved, while policy-based designs remain workable for a small fixed pair of networks. Dynamic routing can run across route-based tunnels, but the underlay still needs independent reachability between public endpoints.

19.5 IPsec and WireGuard

IPsec is a family of protocols that protects IP traffic. In tunnel mode it protects a complete inner packet and adds a new outer IP header. IKEv2 commonly negotiates peer identity, algorithms, keys, and security-association lifetimes. NAT traversal places protected traffic inside UDP, commonly port 4500, so ordinary NAT devices can maintain a mapping.

WireGuard uses public-key peers and UDP with a compact configuration model. Each peer has a public key, endpoint, and allowed IP prefixes. The allowed prefixes influence both routing and which inner addresses are accepted from that peer. Authenticated traffic can update the observed endpoint, supporting roaming between Wi-Fi and mobile networks. Keepalives can preserve NAT mappings when a peer remains silent.

Other VPNs use TLS-oriented transports. Carrying inner TCP through an outer ordered TCP tunnel can perform poorly during loss because both layers retransmit and preserve order. UDP-based outer transport normally avoids that duplicated stream behavior.

19.6 Addressing, DNS, and Overlap

A cryptographic handshake does not make the private network usable by itself. Clients still need correct virtual addresses, protected routes, return routes, firewall permission, and DNS.

Split DNS can send company domains to an internal resolver while other names use the local resolver. If an application independently uses an encrypted public resolver, internal names may fail. Replacing all DNS with a company resolver can create the opposite problem when the tunnel drops.

Overlapping private prefixes create direct ambiguity. A home network and company network both using 192.168.1.0/24 cannot be distinguished by destination alone. Better address planning, translation, application proxies, or non-overlapping IPv6 prefixes are more reliable than trying to route two identical networks.

19.7 Underlay, Overlay, and Access Scope

The physical and routed network carrying tunnel packets is the underlay. The logical network created by the tunnel is the overlay.

Underlay

-> routes between tunnel endpoints

Overlay

-> private prefixes and virtual topology

The overlay can appear as one direct logical link while the underlay crosses many routers. It does not escape underlay latency, loss, bandwidth, MTU, or routing changes. This same model appears in multi-host container and cloud networks.

Traditional VPNs often grant network-level routes. Application-access gateways can instead expose selected services according to user, device, and workload identity without publishing broad internal prefixes. This does not eliminate networking; it moves the boundary from subnet reachability toward an authenticated proxy.

19.8 Failure Patterns and Inspection

A tunnel can report connected while private services fail because protected routes, return routes, DNS, firewall permissions, or traffic selectors are wrong. Only some subnets working suggests a missing route or policy. Small transfers working while large ones stall suggests MTU failure. Disconnection during a network change suggests that the outer endpoint or NAT mapping could not migrate.

`ip route get` can verify that protected destinations use the tunnel while the public peer uses the underlay. Protocol-specific tools show peer endpoints, latest handshakes, allowed prefixes, and byte counters. Capturing on the physical interface shows encrypted outer traffic; capturing on the tunnel interface shows inner traffic before encryption or after decryption. This two-view comparison is especially useful when the outer packets reach the peer but the inner packets never appear, or when inner replies exist but no encrypted return traffic leaves.

19.9 What to Keep in Mind

A tunnel carries an inner packet inside an outer packet. The inner addresses describe logical communication, while the outer addresses deliver traffic between tunnel endpoints through the underlay. VPNs add encryption, integrity, and authentication to this encapsulation. Remote-access VPNs connect devices, site-to-site VPNs connect networks, and split tunneling sends only selected prefixes through the overlay. Routes, return paths, DNS, MTU, NAT traversal, and overlapping private address space remain the most common failure boundaries.

20. Cloud, Virtual Machine, and Container Networking

Cloud platforms and container systems implement familiar networking functions through software. Interfaces, addresses, subnets, routes, firewalls, NAT, DNS, tunnels, load balancers, and sockets still exist; the control plane stores and programs them through APIs rather than only through physical devices.

The useful approach is to translate each virtual abstraction back into a responsibility already established in the book.

20.1 Virtual Machines and Virtual Networks

A virtual machine receives a virtual network interface connected to a software switch in the host or provider platform.

Virtual machine

- > virtual NIC
- > virtual switch
- > physical or overlay network

Inside the guest, the interface owns addresses, routes, neighbors, and sockets like a physical interface. The platform can enforce anti-spoofing, VLAN membership, traffic mirroring, and firewall policy before a packet reaches the guest. A missing packet in a guest capture may therefore have been dropped by the virtual network rather than by the guest firewall.

Cloud providers expose isolated virtual networks containing subnets. Instances receive addresses from those subnets, and route tables choose the next function.

10.20.0.0/16

- > local virtual network

0.0.0.0/0

- > Internet gateway, NAT gateway, firewall, or transit hub

Longest-prefix matching still applies. The provider may answer ARP or Neighbor Discovery on behalf of distant virtual interfaces and may restrict broadcast or multicast, but the guest still experiences a local interface and routed destinations.

20.2 Public and Private Cloud Connectivity

A cloud instance commonly keeps a private address while the platform associates a public endpoint at the edge. Inbound reachability requires a public address or load balancer, a route through the Internet gateway, cloud firewall permission, host policy, and a listening service.

A private subnet lacks direct public publication and commonly sends outbound traffic through a managed NAT gateway. The NAT gateway provides egress translation; it is not a load balancer and does not expose the application.

Internet

- > public load balancer
- > private application instances
- > private database

This architecture narrows the public boundary. Application and database instances need no direct public addresses.

Security groups or equivalent controls are stateful firewalls attached to interfaces or workloads. Rules can permit traffic from an address prefix or from another security group. Some platforms also provide stateless subnet access lists. Traffic must satisfy every applicable layer, including explicit return rules in stateless policy.

20.3 Peering, Transit, and Hybrid Routes

Peering connects two virtual networks privately. Both sides need routes, non-overlapping prefixes, and compatible firewall policy. Peering is commonly non-transitive: if A peers with B and B peers with C, A does not automatically reach C through B.

A transit hub or virtual router connects many networks, VPNs, and private circuits. It scales better than a full mesh and can propagate routes dynamically, but it becomes a central policy and failure boundary that needs redundancy and observability.

Hybrid connectivity joins cloud and on-premises networks through a VPN or private provider circuit. Forward and return routes must exist on both sides, DNS forwarding must have clear authority, and overlapping private ranges remain a major limitation.

20.4 Load Balancers and Private Services

Managed load balancers expose one public or private endpoint and distribute traffic across healthy instances. A layer-4 service forwards TCP or UDP, while a layer-7 service terminates HTTP and can route by host and path.

```
api.example.com/orders
-> order service
api.example.com/products
-> product service
```

Internal load balancers publish private services to connected networks without exposing them to the Internet. Private DNS can map stable service names to these endpoints. Managed cloud services can also expose private endpoints inside a virtual network so object storage, databases, or APIs are reached through private addresses rather than public egress. DNS commonly returns the private answer only inside associated networks, which means a correct name can still be unreachable from a client lacking the corresponding route.

A virtual machine used as a router, NAT appliance, firewall, or VPN gateway may require both platform-level forwarding permission and guest operating-system forwarding. Route tables, source/destination checks, guest firewall policy, and kernel forwarding must all agree.

20.5 Container Network Namespaces

Containers commonly run inside separate network namespaces. Each namespace has its own interfaces, addresses, routes, neighbor table, firewall state, and sockets.

A virtual Ethernet pair connects the container namespace to the host. One endpoint appears as `eth0` inside the container and the other appears on the host, often attached to a software bridge.

```
Container A
-> veth
-> bridge
-> veth
-> Container B
```

Containers on one bridge can exchange local frames when policy permits. The host routes between the container subnet and other networks. Outbound traffic commonly uses host NAT, while publishing a host port creates destination translation toward the container.

```
Host 0.0.0.0:8080
-> container 172.18.0.2:80
```

Binding the application only to container loopback still prevents the mapping from reaching it. Declaring a port in container metadata also does not necessarily publish it on the host.

20.6 Container DNS and Multi-Node Networking

Container platforms provide internal DNS so applications can use stable service names instead of temporary instance addresses. On one host, names may map directly to container addresses. In an orchestrated cluster, a service name can map to a virtual service address or to a dynamic set of ready workloads.

Across nodes, the cluster must make workload addresses reachable through ordinary routes, BGP distribution, cloud-native interfaces, or an overlay tunnel.

```
Inner packet:
  pod A -> pod B
Outer packet:
  node A -> node B
```

The same underlay-overlay model from VPNs applies. Node reachability, tunnel MTU, and encapsulation policy affect workload traffic even when the application sees direct pod addresses.

Containers in one pod commonly share a network namespace, loopback interface, IP address, and port space. Different pods receive distinct addresses.

20.7 Services, Ingress, Readiness, and Service Meshes

Workload instances are temporary, so a service provides a stable logical endpoint and selects ready backends.

Service address: 10.96.0.25:8080

Ready backends:

10.244.1.5:8080

10.244.2.7:8080

The forwarding implementation may use kernel NAT, virtual-server rules, eBPF, a user-space proxy, or a cloud load balancer. Readiness controls membership: a new workload should not receive traffic until initialization succeeds, and a terminating workload should be removed before it exits. A workload address can remain reachable while the service correctly excludes it.

Ingress exposes HTTP services from outside the cluster, commonly through a cloud load balancer and an ingress proxy. It can terminate TLS and route by host and path. Internal hops use HTTP, HTTPS, or mTLS according to trust boundaries.

A service mesh adds proxies beside or near workloads to provide service identity, mTLS, traffic splitting, metrics, tracing, retries, and timeouts. It does not replace routes, DNS, sockets, or application semantics. Instead, it adds another connection and policy boundary. Client, gateway, mesh, and application timeout and retry behavior must therefore be coordinated to avoid duplicate work and retry storms.

20.8 One Combined Request Path

A browser can reach a cloud-hosted container application through:

Public DNS

-> cloud load balancer

-> cluster ingress

-> service

-> ready workload

The public load balancer accepts or terminates TLS and selects an ingress target. The ingress routes the HTTP authority and path to a service. The service selects a ready workload. If that workload runs on another node, the cluster routes or encapsulates the packet between nodes before delivering it into the destination namespace and listening socket.

A public 502 can mean that the edge is healthy while the ingress has no reachable backend. A service name can resolve while its endpoint set is empty. A workload can listen correctly while network policy blocks the source. A private endpoint can resolve correctly but remain unreachable because the client has no matching route.

Every abstraction reduces to familiar questions: which name was resolved, which address and route were selected, which policy permitted the packet, which intermediary selected a backend, and which socket accepted it?

Cloud flow logs, load-balancer access logs, NAT metrics, DNS query logs, service endpoint lists, and network-policy decisions provide evidence outside the guest. A packet absent from `tcpdump` inside a virtual machine or container may have been rejected earlier by the provider, node, or service layer.

20.9 What to Keep in Mind

Cloud and container networking implement familiar functions through software. Virtual machines have interfaces and routes; virtual networks have subnets, gateways, firewalls, peering, transit, NAT, and load balancers. Containers use namespaces, virtual Ethernet pairs, bridges, NAT, DNS, and stable service abstractions. Multi-node clusters route or tunnel workload traffic, ingress publishes HTTP services, and meshes add proxy-based identity and policy. Virtualization changes the control plane and packet path, but successful communication still requires correct naming, routes, return paths, security rules, and listening sockets.

21. Troubleshooting Networks Systematically

Network troubleshooting becomes manageable when the path is divided into stages and evidence is used to eliminate what already works. The goal is not to run every command in a fixed order. It is to find the last proven boundary and the first unproven one.

Link

- > address
- > route
- > neighbor
- > DNS
- > transport
- > TLS
- > HTTP
- > application

A known HTTP 500 already proves that the earlier stages reached an application path. Beginning with cable tests would add noise rather than information.

21.1 Record the Exact Symptom

Record the operation, hostname, selected network, address family, protocol, port, time, duration, and exact error before repeated experiments change the evidence. Compare one working and one failing case.

Works by IP, fails by name

- > DNS or address selection

Works on mobile data, fails on Wi-Fi

- > local network or provider path

Works through IPv4, fails through IPv6

- > route, firewall, or listener for one family

Small traffic works, large transfers stall

- > MTU, buffering, body limit, or timeout

Only one backend address fails

- > unhealthy distributed endpoint

Changing one variable is often more useful than collecting a large undifferentiated log.

21.2 Interface, Address, Route, and Neighbor

On Linux, begin with `ip -brief address` and `ip link show`. Check that the intended interface is up, owns a plausible address, and uses the expected prefix. `169.254.x.x` commonly indicates failed IPv4 DHCP. An IPv6 interface with only a link-local address may lack a usable Router Advertisement.

Ask the routing table for the exact decision:

```
ip route get 203.0.113.25
ip -6 route get 2001:db8:100::25
```

The result reveals the selected interface, gateway, and source address. On a multi-homed system, the wrong source can cause reply routing or firewall problems.

Then inspect the local next hop with `ip neighbor` or `ip -6 neighbor`. An incomplete or failed gateway entry suggests wrong VLAN, prefix, gateway state, or neighbor discovery. A remote Internet server should not normally appear as a local neighbor; the host resolves the gateway's link address.

`ping` can measure ICMP reachability and round-trip time, but failure is not conclusive because a firewall can block echo while the intended service works. Success proves only that ICMP completed, not that a TCP port, TLS identity, or application is healthy.

21.3 DNS as a Separate Stage

Use the normal resolver path:

```
getent ahosts shop.example.com
```

Use `dig` to inspect record types, TTLs, delegation, or a chosen resolver. Note whether split DNS, VPN policy, a container resolver, or an encrypted application resolver changes the answer.

Test a web endpoint at a specific address while preserving the hostname:

```
curl --resolve shop.example.com:443:203.0.113.25 \
  https://shop.example.com/
```

If this works while normal access fails, DNS or address selection is likely responsible. Connecting directly to the IP is not equivalent because TLS SNI and HTTP authority may select a different certificate or virtual host.

21.4 Transport, Sockets, TLS, and HTTP

A TCP test asks whether the destination accepts one port.

```
nc -vz shop.example.com 443
```

A successful connection proves that a TCP handshake completed. **Connection refused** usually means a host or intermediary actively rejected the port. A timeout means expected progress did not arrive before the local deadline; routing, silent filtering, server state, or the return path can all be responsible.

On the server, `ss -ltnup` confirms which address and port a process actually listens on. A service bound only to `127.0.0.1` can work locally while remaining unavailable remotely.

Inspect TLS with:

```
openssl s_client \
  -connect shop.example.com:443 \
  -servername shop.example.com
```

The command reveals the presented chain, name, validity, negotiated protocol, and verification result. `curl -v` continues through HTTP and exposes redirects, headers, status codes, and proxy behavior.

A 401 or 403 proves that an HTTP endpoint processed the request and denied identity or permission. 502 and 504 prove that a proxy was reached but its upstream failed or exceeded a deadline.

21.5 Measure the Request Stages

A single total duration hides where time was spent. `curl` can report DNS, connection, TLS, first-byte, and total time.

Slow DNS

-> resolver or delegation

Slow connect

-> path, filtering, listener, or overload

Slow TLS

-> handshake, certificate, or CPU

Slow first byte

-> proxy queue or application work

Slow body

-> bandwidth, loss, flow control, or storage

Client telemetry should retain the same stage separation. Server and proxy logs should record request identifiers, upstream selection, dependency timing, cancellation, and retry count. Logs describe individual events, metrics reveal rates and trends, and traces connect one logical request across several processes. None replaces packet capture: traces show instrumented application work, while capture shows what crossed one interface.

21.6 Path Observation and Packet Capture

`traceroute` or `tracpath` reveals routers that return hop-limit expiration messages. Asterisks do not necessarily mean ordinary traffic is lost; routers can forward packets while suppressing diagnostic replies, and equal-cost routing can expose different paths.

Path MTU problems often produce a distinctive progression: neighbor discovery works, the transport handshake succeeds, small requests work, and larger bodies stall. `tracpath`, packet capture, and ICMP Packet Too Big messages help identify the boundary. Tunnels and overlays deserve attention because their extra headers reduce the usable inner MTU.

Packet capture shows what crossed one interface.

```
sudo tcpdump -ni any host 203.0.113.25
sudo tcpdump -ni any tcp port 443
sudo tcpdump -ni any port 53
```

Repeated SYN packets with no reply suggest silent loss or a broken return path. SYN followed by RST means the endpoint was reached but the port was rejected. A completed handshake followed by a TLS alert moves the failure boundary upward. Retransmissions show missing acknowledgment progress but do not by themselves prove whether the cause is loss, delay, overload, or an incomplete capture.

Capture at both sides of the smallest uncertain boundary. In virtual systems that can mean the guest interface, host veth, bridge, node interface, overlay tunnel, service proxy, or remote workload. Platform flow logs may expose a policy drop that never appears inside the guest.

21.7 Firewalls, NAT, VPNs, and Virtual Networks

For a published private service, verify the entire chain:

```
Public route
-> edge firewall
-> NAT or load balancer
-> internal route
-> host or workload policy
-> listening socket
-> return route
```

A packet can reach the server while the reply bypasses NAT or stateful firewall state. One-way captures are especially useful for finding this asymmetry.

For a VPN, verify both routes: protected destinations should use the tunnel, while the public peer must remain reachable through the underlay. For cloud networks, compare route tables, security groups, subnet policy, flow logs, load-balancer health, and guest state. For containers, verify DNS, service endpoints, readiness, network policy, node routes, and overlay state.

Virtualization changes where evidence is stored, not the questions being asked.

21.8 Intermittent and Performance Problems

For intermittent failure, record the resolved address, address family, selected backend, HTTP version, source network, status, timing, and trace identifier for successful and failed samples. One bad DNS address or load-balancer target often appears random.

Separate latency from throughput. A small request measures setup and processing delay; a large transfer also exercises available bandwidth, congestion control, flow control, compression, and storage. Saturated links can maintain high throughput while queueing creates severe latency, a condition called bufferbloat.

Loss affects protocols differently. TCP retransmits and may reduce its sending rate. Real-time UDP applications may skip late media and expose audible or visible gaps. Jitter buffers trade extra delay for smoother playback.

21.9 Preserve Evidence and Change One Thing

Do not begin by disabling every firewall, deleting routes, clearing all caches, or restarting all systems. These actions destroy evidence, create exposure, and can make a transient issue disappear without revealing its cause.

Prefer:

```
Record symptom and time
-> compare working and failing cases
-> inspect state and counters
-> capture around one boundary
-> make one controlled change
-> verify and reverse when needed
```

Distributed evidence needs synchronized clocks and shared identifiers. Use wall-clock time with an explicit timezone or UTC for correlation and monotonic clocks for durations. A clock error can also break certificate

validation, token expiration, signed requests, and security protocols, so time synchronization is part of networked-system correctness rather than merely log formatting. Do not log passwords, bearer tokens, cookies, private keys, or sensitive bodies merely to improve diagnostics.

21.10 A Compact Workflow and Worked Examples

1. Record the exact operation, error, and timestamp.
2. Compare one working and one failing case.
3. Verify interface, address, prefix, and selected source.
4. Ask the routing table for the actual path and next hop.
5. Check DNS and the chosen IPv4 or IPv6 address.
6. Test the transport port and confirm the listener.
7. Inspect TLS and HTTP as separate stages.
8. Compare proxy, application, and dependency evidence.
9. Capture around the smallest uncertain boundary.
10. Change one variable and preserve the result.

Skip stages already proved by stronger evidence. A certificate mismatch proves that transport reached a TLS endpoint. An HTTP 500 proves that HTTP delivery reached an application path.

Name fails, direct address works. Preserve the intended TLS server name and HTTP authority while connecting to the known address. If that succeeds, the service, transport, and TLS identity are usable; inspect the resolver, returned address family, cached answer, and split-DNS policy.

TCP connects, TLS fails. A successful TCP handshake proves routing and the listening transport endpoint. A name mismatch, expired certificate, missing intermediate, or TLS alert moves investigation to SNI routing, certificate deployment, trust stores, clocks, and protocol compatibility rather than cables or basic IP routes.

Small requests work, large transfers stall. Successful handshakes and small responses prove several lower layers, but not usable packet size along the complete path. Inspect tunnel overhead, Path MTU Discovery, TCP MSS, and blocked ICMP Packet Too Big messages.

Timeout and retry policies are architectural behavior rather than troubleshooting patches. Inner operations should normally fail before outer callers abandon them, cancellation should propagate where work is no longer useful, and one intentional layer should own retries.

21.11 What to Keep in Mind

Troubleshooting should narrow a failure boundary rather than collect unrelated commands. Interface state, addressing, route selection, neighbor resolution, DNS, transport, TLS, HTTP, proxies, tunnels, virtual networks, and application dependencies are separate stages. Exact errors and status codes prove how far the request progressed. Compare working and failing cases, inspect both sides of the smallest uncertain boundary, preserve evidence, and change one variable at a time.

Part VI — Designing Reliable and Secure Networked Systems

22. Designing Reliable and Secure Networked Systems

A network call is not a local method call with a longer execution time. A request can be delayed, duplicated, processed after the caller gives up, or completed successfully while the response is lost. Several intermediaries can remain healthy while one dependency or one direction of the path fails.

Reliable and secure software must therefore define what uncertainty means, how long work may continue, how much load can accumulate, which retries are safe, and which identities may cross each boundary.

22.1 Partial Failure, Deadlines, and Cancellation

Silence cannot reveal whether a request never arrived, is still queued, is being processed, completed successfully, or produced a response that was lost. A timeout converts that uncertainty into a local decision: the caller stops waiting. It does not prove that the remote operation did not occur.

This is why a payment timeout should not automatically be translated into “payment was not taken.” The honest result can be “outcome unknown,” followed by a status lookup or idempotent retry.

Different stages deserve different limits: DNS, connection establishment, TLS, upload, server processing, dependency calls, and response transfer. A total deadline should propagate through the call chain. If a client has ten seconds and a gateway spends two, the service has eight seconds remaining rather than a new ten-second budget.

Cancellation is cooperative. Closing the client socket does not guarantee that a database transaction or remote call stops immediately. Optional work should normally observe cancellation, while a business operation that has crossed a durable commit boundary may need to finish and expose its result later.

22.2 Safe Retries and Idempotency

Retries help with failures that are plausibly temporary, such as one unavailable backend, a reset during deployment, or brief overload. They do not repair invalid requests, denied permission, wrong names, or persistent routing errors.

A retry is safe only when repetition cannot create an unacceptable duplicate effect. Reads are commonly retryable. Creating an order or charging a card may not be: the first attempt can commit while its response is lost.

Applications solve the ambiguity through idempotent design. A client can choose a stable resource identifier such as `PUT /orders/9001`, or attach an idempotency key to a `POST`. The server stores the result associated with that intended business operation and returns the same outcome when the request is repeated.

Retries should remain inside the original deadline and use exponential backoff with random jitter. A retry budget limits additional traffic. Only one deliberate layer should normally own a retry policy; independent retries in the client, gateway, service mesh, application, and database driver can multiply attempts during an outage.

22.3 Protecting Capacity

Reliability includes rejecting or slowing work before queues, memory, connection tables, and worker pools collapse. Every queue should be bounded. A queue absorbs a brief burst, but if work arrives continuously faster than it can be processed, a larger queue only postpones failure while increasing latency.

Backpressure makes overload visible to the producer. TCP, HTTP/2, and QUIC provide transport flow control, while application queues can delay or reject producers. Load shedding deliberately rejects excess or lower-priority work so critical operations remain responsive. A fast `503 Service Unavailable` is often better than accepting every request and allowing all of them to time out.

Connection pools also require limits. Reuse avoids repeated TCP, QUIC, and TLS setup, but an unbounded pool can overwhelm the destination and a pool that is too small creates hidden local queueing. Maximum connections, pending-request limits, idle lifetime, and connection lifetime should reflect client concurrency and server capacity.

Circuit breakers stop ordinary calls after sustained failure and allow limited recovery probes. Bulkheads isolate pools or queues so one slow dependency cannot consume every resource. Rate limits provide fairness, while graceful degradation substitutes cached or reduced functionality only when correctness permits it.

22.4 Durable Delivery and Ordering

Messaging separates accepting work from processing it, but it does not create perfect delivery. At-most-once delivery can lose messages. At-least-once delivery retries and can produce duplicates. “Exactly once” is normally an application outcome built from durable state, unique identifiers, and idempotent effects.

acknowledgments also have levels. A TCP acknowledgment proves that bytes reached the remote transport stack. A broker acknowledgment can prove that a message was stored. An application acknowledgment can prove that a consumer processed it. The protocol must state which guarantee is actually being made.

Consumers should expect duplicate and out-of-order delivery. Stable message identifiers, unique database constraints, entity versions, and sequence numbers let them ignore repeated or stale work. Ordering is usually needed within one entity or workflow rather than across the whole system.

When a service must update its database and publish an event, an outbox can record the state change and outgoing message in one transaction, then publish later. Publication may happen more than once, so consumers remain idempotent. Longer workflows use explicit state and compensating actions rather than pretending that one transaction can roll back independent network services.

22.5 Graceful Recovery and Failure Domains

Health checks should express the decision they control. Liveness asks whether a process should be restarted; readiness asks whether it should receive new traffic. Including every remote dependency in readiness can create a cascading outage in which one shared slowdown removes every application instance.

Shutdown should reverse startup. The instance becomes unready, waits for load balancers and registries to stop new work, drains requests or streams, stops consuming messages, and then exits. Long-lived clients need a recovery model: after reconnecting they may authenticate again, provide the last processed sequence, and receive missed events.

Redundancy matters only when replicas occupy independent failure domains. Two instances on one host do not survive host failure; two links in one conduit do not survive one cut; two regions managed by one mistaken deployment can still fail together. Designs should identify shared power, switches, zones, DNS, certificate paths, databases, configuration, and operator actions.

Failure behavior should be tested deliberately. Controlled experiments can introduce delay, packet loss, one unavailable backend, a full queue, an expired certificate, or a network partition and verify a specific expected response. Observability should distinguish original work from retries and expose queue age, connection-pool wait, circuit state, cancellation, and fallback use.

22.6 Communication Requirements and Trust Boundaries

Secure design begins with required communication rather than with products or VLAN numbers. For each path, identify the source, destination, protocol, port, direction, identity, data sensitivity, and availability requirement.

A trust boundary exists wherever identity, exposure, or policy changes: the Internet edge, user-to-server path, application-to-database path, management network, VPN gateway, or connection to an industrial system. Crossing it should trigger explicit decisions about authentication, authorization, encryption, validation, logging, rate limiting, and failure behavior.

Least privilege grants only the path required for a function. Public clients can reach HTTPS on the edge, the edge can reach the application port, and the application identity can reach the database port. Users do not need direct database reachability, and the database does not need a direct Internet route. Default-deny policy keeps newly opened ports unreachable until intentionally reviewed.

Network location complements but does not replace identity. Services should authenticate through workload identity, mTLS, signed tokens, or another scoped mechanism. Secrets and keys should come from protected storage, rotate automatically, and remain absent from source code, images, URLs, and logs.

22.7 Secure Intermediaries and Egress

Proxies and protocol parsers form security boundaries. The edge should accept only expected HTTP authorities, replace untrusted forwarding headers, enforce size and timeout limits, and reject ambiguous message framing. Applications should trust **Forwarded** or **X-Forwarded-*** values only from approved proxies.

Outbound communication also needs policy. A feature that fetches arbitrary user-provided URLs can become Server-Side Request Forgery and reach loopback, link-local metadata, or private administration services. Destination allowlists, address validation after DNS and redirects, and egress firewalls provide complementary protection.

Every intermediary should justify its existence. CDNs, gateways, service meshes, proxies, and load balancers can add security, caching, routing, or resilience, but each also adds a connection pool, timeout, retry policy, log, and failure mode. The simplest useful architecture is the smallest one that satisfies the required boundaries and availability.

Address plans should use non-overlapping hierarchical prefixes with room for growth. Repeated private ranges complicate VPNs, cloud peering, mergers, and diagnosis, while aggregatable allocations simplify routes. IPv6 needs the same firewall, monitoring, and testing attention as IPv4; dual stack creates two operational paths rather than one primary path and one harmless extra.

Keep layer-2 domains bounded and route between meaningful security or failure zones. Segmentation should follow different communication requirements or risk, not every minor organizational distinction. Redundant links, instances, and regions should avoid hidden shared dependencies and retain enough spare capacity to serve traffic when one member is unavailable.

22.8 One Secure and Reliable Application

Consider a public order service:

Internet

- > protected HTTPS edge
- > private application instances
- > private database

Public DNS names only the edge. The edge validates TLS and authority, limits body size and rate, replaces forwarding headers, and sends traffic only to ready instances. Application instances have no public addresses, use workload identities, and reach approved external services through controlled egress. The database accepts its service port only from the application path and has no direct Internet route.

The order API applies a total deadline and an idempotency key. It commits the order and an outbox record together. A lost response can be retried safely, publication may occur more than once, and consumers use unique identifiers to prevent duplicate effects. Optional dependencies fail through circuit breakers and degraded responses, while critical uncertainty produces an accurate pending or unavailable state rather than invented success.

Every connection exists because one documented actor needs one documented operation, and every boundary provides enough evidence to distinguish routing, policy, application, and dependency failure.

Software-defined routes, firewall rules, DNS records, load balancers, certificates, and VPNs should be versioned, reviewed, and validated like application code. A safe change has staged deployment, success checks, rollback criteria, and a management path that does not depend entirely on the configuration being changed. Synthetic tests should resolve the real name, establish the intended secure transport, send a representative request, and verify the response from the network perspectives that matter.

22.9 What to Keep in Mind

Network calls can fail partially and ambiguously, so deadlines, cancellation, retries, and idempotency must be designed together. Backoff, jitter, bounded queues, circuit breakers, bulkheads, rate limits, and load shedding protect capacity during failure. Durable messaging requires explicit duplicate, ordering, acknowledgment, and recovery rules. Security begins with communication requirements, trust boundaries, least privilege, service identity, strict proxy handling, and controlled egress. Reliable and secure behavior belongs to the complete path, not to one protocol or appliance.

23. A Durable Mental Model for Networks

Networking becomes easier when each mechanism is remembered as an answer to one question rather than as an isolated protocol.

Physical medium

-> How can information travel?

Frames

-> Which nearby interface should receive it?

IP

-> Which network destination should receive it?

Routing

-> Which next hop moves it closer?

Transport

-> Which process and communication behavior are required?

Secure transport

-> Is the connection authenticated and protected?

Application protocol

-> What useful operation is being requested?

23.1 One Communication, Several Views

For `https://api.example.com/orders/42`, the application sees a URL and an HTTP operation, DNS sees a name, the operating system sees an address and port, the routing table sees a destination prefix, and the local link sees the next hop's MAC address. These are not competing descriptions. Each is the identifier needed at one scope.

A domain name provides a stable application-facing identity; an IP address provides a routed location; a port selects a transport endpoint; a MAC address delivers one local frame; a certificate proves a cryptographic service identity; and a cookie or token represents application state or caller identity. Confusion begins when one identifier is expected to solve another layer's problem.

23.2 Decisions and Boundaries

Ask which header or state a device examines. A switch uses a destination MAC address, a router uses an IP prefix, a stateful firewall or NAT also considers ports and connection state, a load balancer may use transport or HTTP information, and the application uses the method, path, headers, identity, and body. One physical appliance can perform several roles, so the active decision matters more than the product name.

Frames cross local links; packets cross routed networks; transport sessions join sockets; proxies terminate one connection and create another; tunnels place an inner packet inside an outer path. At each routed hop the local frame changes, while the packet continues toward its destination unless NAT intentionally rewrites it. Reachability requires a usable forward and return path, and stateful devices may require both directions to cross the same state.

23.3 Encapsulation, State, and Time

Encapsulation adds the context required at each scope:

Application operation

-> HTTP or another protocol

-> TLS over TCP, or TLS integrated into QUIC

-> IPv4 or IPv6 packet

-> Ethernet or Wi-Fi frame

-> physical signal

The stack is a set of responsibilities rather than one rigid arrangement. HTTP/1.1 and HTTP/2 commonly use TLS over TCP; HTTP/3 uses TLS integrated into QUIC over UDP.

Reliability and security exist at several scopes. A Wi-Fi acknowledgment confirms one radio frame, TCP confirms received byte ranges, HTTP reports an application response, and business completion may require a durable database result. Wi-Fi encryption protects one local hop, a VPN protects tunnel endpoints, TLS protects one secure transport session, and application authorization decides which operation an identity may perform.

Caches make time part of the system. DNS, neighbor tables, routes, HTTP representations, TLS sessions, health checks, and service discovery can remain temporarily valid after authoritative state changes. A configuration change is complete only after the relevant cached state expires or is invalidated.

23.4 Reading Failures as Evidence

An error often proves how far the operation progressed. A name-resolution failure occurs before transport. A refused TCP connection proves that a host or intermediary rejected the port. A certificate mismatch proves that the client reached a TLS endpoint. 401, 403, and 404 prove that an HTTP endpoint processed the request. 502 and 504 prove that a proxy was reached but its upstream failed or exceeded a deadline. 500 places the failure in an application path.

Troubleshooting therefore asks two questions: what is the last stage known to work, and what is the first stage known to fail? Address, route, neighbor, DNS, socket, TLS, HTTP, proxy, application, and dependency evidence each describe a different boundary. Compare a working and failing case and stop at the smallest uncertain one.

23.5 Ambiguous Outcomes and Explicit Design

A client timeout does not prove that the server performed no work. The request may have committed while the response was lost. Safe systems use deadlines, cancellation, stable resource identifiers, idempotency keys, status queries, and duplicate detection to turn that ambiguity into deliberate behavior.

The same discipline applies to architecture. Document required communication, route only necessary prefixes, permit only necessary identities and ports, define where TLS and proxies terminate, bound every queue and connection pool, and provide evidence at each important boundary. Virtual networks, containers, cloud gateways, VPNs, and service meshes still reduce to interfaces, addresses, routes, policies, transports, names, and application state.

23.6 The Complete Path

User action

- > application creates an operation
- > application protocol describes it
- > secure transport authenticates and protects it
- > ports and sockets identify endpoints
- > IP routes packets toward a destination
- > Ethernet or Wi-Fi delivers each local step
- > the physical medium carries signals
- > intermediaries switch, route, filter, translate, proxy, and balance
- > the destination reverses the responsibilities and performs the operation
- > the response returns, not necessarily through the same physical path

Technologies will change, but five questions remain stable:

- Which identity is being used?
- Which layer or scope is deciding?
- Which boundary is being crossed?
- Which state is being remembered?
- What evidence proves progress?

Once those questions can be answered, unfamiliar network systems become variations of a model already understood.

23.7 What to Keep in Mind

A network is a sequence of cooperating scopes rather than one opaque pipe. Signals cross media, frames cross local links, packets cross routed networks, transports connect sockets, secure sessions protect endpoints, and application protocols describe useful operations. Design and troubleshooting both become manageable when the active identifier, decision, boundary, state, and last proven stage are explicit.